

Problem Solving Strategies and Computational Approaches SCS1304

Handout 1 : Introduction to Computational Thinking

Prof Prasad Wimalaratne PhD(Salford),SMIEEE

Overview

- This course introduces students to the techniques and methodologies for analyzing and solving complex problems in a structured manner by equipping them with the skills necessary to effectively analyze system requirements, identify problems, and devise solutions using structured approaches.
- CS1304 aims to lay sound a foundation or designing algorithms for real world problems using pseudocodes and designing flowcharts (i.e without any coding)

Indicative Content

- **Introduction to Computational Thinking**

- ❖ Decomposition, Pattern Recognition, Abstraction, Algorithmic Thinking
- ❖ Linear Search, Binary Search, Fibonacci Sequence, Introduction to Recursion, Types of Recursion (Linear Recursion, Nested Recursion, Tail Recursion, Tree Recursion, Head Recursion & Indirect Recursion etc.), Applications of Recursion, Best Practices and Pitfalls in Recursive Algorithms, Recursion vs Loops/Iteration

- **Asymptotic Notation**

- ❖ Analysis of Algorithms, Order of Algorithms, Practical Considerations, Every-case time complexity, Worst-case time complexity, Best-case time complexity, Average-case time complexity.

Indicative Content ctd..

- **Problem-Solving Strategies**

- ❖ Typical Strategies, Trial and Error, Algorithm and Heuristic, Means-Ends Analysis, The problem- solving Process
- ❖ Brute Force: Origins of "Brute Force" approach, Brute Force in Computing, Brute Force Algorithms, Time Complexities of Brute Force Algorithms, Advantages and Disadvantages, Optimizing Brute Force, Alternatives to Brute Force, Brute Force Applications
- ❖ Divide and Conquer: Merge Sort, Quick Sort, Strassen's Matrix Multiplication, Further examples of Divide and Conquer Algorithms.

- **Dynamic Programming**

- ❖ Introduction to Dynamic Programming (DP), Divide Conquer vs Dynamic Programming, Examples of DP, Applicability of DP in solving problems, Approaches to DP – Tabulation & Memoization, Further examples of DP.

Indicative Content ctd..

- **Greedy Algorithms**

- ❖ Coin Change Problem, Greedy vs Not Greedy Algorithms, Activity Selection Problem, dynamic programming vs Greedy Approach, Knapsack Problem, Spanning Trees, Algorithms to find Minimum Spanning Tree, Kruskal's Minimum Spanning Tree, Algorithm, Prim's Minimum Spanning Tree Algorithm, Further examples of Greedy algorithms.

- **Backtracking**

- ❖ Maze Problem, State Space Trees, Backtracking Applications, Sudoku, N-queen problem using Backtracking, Hamiltonian Path, Hamiltonian Cycle using Backtracking, Optimizing backtracking, Further examples of Backtracking algorithms.

- **Branch Bound**

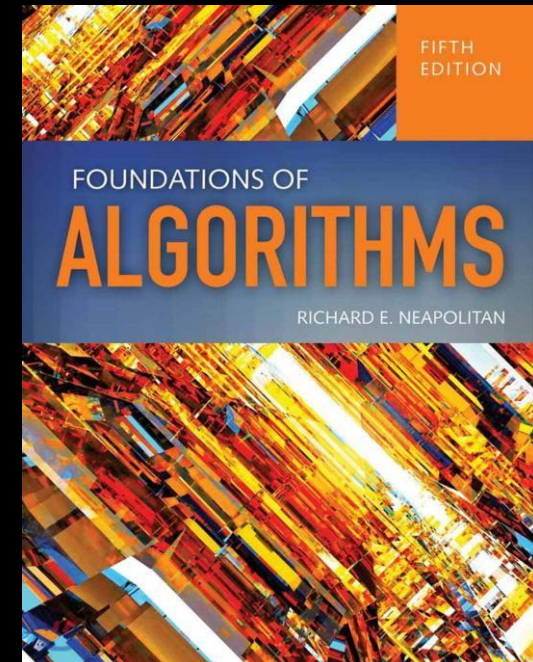
- Traveling Salesman Problem, Branch Bound (FIFO, LIFO, Least Cost-Branch and Bound), Further examples of Branch Bound algorithms

Course Objectives

- to provide students with a basic of understanding of how to write and analyze algorithms.
- to impart to them the skills needed to write algorithms using the standard algorithm design strategies.
- to provide a sound foundation required for developing strong software engineering skills.
- to provide an understanding of some common algorithms, as well as some general approaches to developing algorithms.
- to develop skills to device a solution to a computational problem by answering to a problem, but the best answer.
- to be able to evaluate an algorithm and analyze how its performance is affected by the size of the input so that one can choose the best algorithm to solve a given problem.

Course Overview

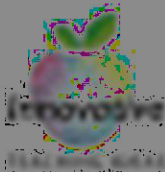
- Credits : 2C (2L + 0P)
Rubric:
 - 60% Exam, 40% Assignments (Supervised Quizzes)
- Delivery: 2-hour Lecture + **2-hour Tutorials per week (important!)**
- References: BOOKS and online Materials
Neapolitan, R. and Naimipour, K., 2015.
Foundations of Algorithms. 5th Edition,
Jones & Bartlett Publishers.
<https://github.com/davidkmw0810/algorithm/tree/master/knap>



Handout 1: Overview

- Introduction to Computational thinking
 - decomposition, pattern recognition, abstraction, and algorithm design.
- Significance of understanding each and their roles in computational problem solving

What is Computational Thinking?



Computational thinking is a set of skills and processes that enable one to navigate complex problems

Refer

- **Computational Thinking**
- <https://www.youtube.com/watch?v=dHWmnayy8MY&t=15s>
- **Solving Problems at Google Using Computational Thinking**
- <https://www.youtube.com/watch?v=SVVB5RQfYxk>
- **Ref: Computational Thinking Defined**
- <https://towardsdatascience.com/computational-thinking-defined-7806ffc70f5e>

Computer Science & Problem Solving

- Computer science is the discipline of how to **solve problems using computers**
- We **aim for efficient, general solutions** that will work on a **wide variety of problems**
- Although computer science has existed as a field for about 70 years, its **roots in mathematics and computation go back thousands of years!**
- CS is a field that relies **partly on old mathematical ideas** but experiences advances in development of **new techniques at an extraordinary pace**

Computational Thinking ?

- helps one to think logically and clearly, whether you're writing code, solving math problems, or planning something in everyday life
- Approaching a problem in a systematic manner
- Creating and describing a solution to a problem
- Work out how a computer or a person can solve a problem

Computational Thinking ?

1. Breaking a big problem into smaller sub-problems (this is called decomposition).
2. Finding patterns in the problem that repeat. i.e Identifying similarities, trends, or repeated patterns among problems.
3. Ignoring unnecessary details and focusing on what's important (abstraction).
4. Creating step-by-step instructions to solve the problem (algorithms). i.e Developing a clear sequence of steps to solve a problem efficiently.

What is Computational Thinking

- Problem Solving?
 - Computational thinking is a **set of skills and processes** that **guide problem solving**.
 - What makes this especially **different from other problem-solving processes** is that it, **in the end, results in an algorithm**, which is a series of steps **a person or computer uses to perform a task or solve a problem**.
 - Computational thinking is derived from the process computer scientists use to develop code and communicate with computers through algorithms

Significance of Computational Thinking Skills

- Computational thinking allows us to take a **complex problem, understand** what the problem is and **develop possible solutions**.
- Since there are **more than one potential solution** to a given problem **evaluation of different solutions** is also important

e.g. sorting of given set of items according to alphabetical or numerical order

e.g Bubble sort Algo ->

<https://www.youtube.com/watch?v=9I2oOAr2okY>

- We can then present these solutions in a way that a computer, a human, or both, can understand.

Significance of Computational Thinking Skills ctd..

- Computational thinking is a problem-solving process that involves **looking at possible solutions abstractly and algorithmically**, in a series of ordered steps.
- Without the **ability to problem-solve using computational thinking**, it would be **difficult to define and replicate innovative solutions** to modern computing application

What is computational thinking? Ctd..

- How computer scientists think – **how they reason and work through problems**
- Computer science encompasses **many sub-disciplines** that support the goal of **solving problems**:
 - Computer theory areas: these are the heart and soul of computer science
 - Algorithms, Data structures
 - Computer systems areas
 - Hardware design, Operating systems, Networks
 - Computer software and applications
 - Software engineering, Programming languages, Databases, Artificial intelligence, Computer graphics

A major goal of this course is to help you develop your computational thinking and problem-solving skills

Computational Thinking is Coding?

- Not quite!. While **computational thinking is the problem-solving process that can lead to code**, coding is the process of programming different digital tools with algorithms.
- Coding is a means to apply solutions **developed through the processes of computational thinking**.
- **Algorithms**, in the case of coding, are a series of **logic-based steps that communicate with digital tools and help them execute different actions**.
- However, computational thinking results in **algorithms for both computers and people**, making it much more broadly applied **with and without technology**.
- At its core, the **steps of the computational thinking process** enable people to tackle large and small problems

What is Computational Thinking?

- A way of thinking for logically and methodically solving problems
 - E.g., purposeful, describable, replicable
- Requires skills such as
 - Decomposition
 - Pattern Recognition
 - Abstraction
 - Generalization
 - Algorithm Design
 - Evaluation



Computational Thinking Steps

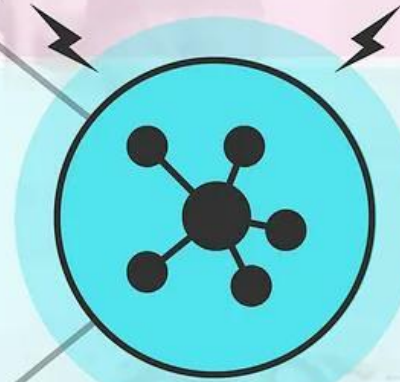


Problem Specification

- **Abstraction:** Removing aspects of a problem that are not needed for its solution
- **Decomposition:** Breaking a big problem down into smaller, more manageable sub-problems
- **Pattern Recognition:** Analyzing any kind of repeating elements or sequences in the problem

Algorithmic Expression

- **Algorithm Design:** Creating step-by-step instructions for solving the problem



Solution Implementation & Evaluation

- **Generalization:** Extending a solution for a particular problem to other kinds of problems

Computational Thinking

1. **Problem Specification:** We start by **analyzing the problem**, stating it precisely, and establishing the criteria for the solution.

A computational thinking approach to a solution often starts by **breaking complex problems** down into more familiar or **manageable sub-problems**, sometimes called problem **decomposition**, frequently using deductive or probabilistic reasoning.

This can also involve the ideas of **abstraction** and **pattern recognition**.

Computational Thinking ctd..

2. **Algorithmic Expression**: We then need to find (a) an algorithm, a precise sequence of steps, that solves the problem using (b) appropriate data representations.

This process uses **inductive thinking** and is needed for transferring a particular problem to a **larger class of similar problems**.

This step is also sometimes **called algorithmic thinking**. We can **further break it down** into either (a) **imperative**, like procedural or modular, and (b) **declarative**, like functional, **approaches to algorithmic solutions**.

What is the difference between declarative and imperative programming?

- Declarative programming is often simpler and more efficient, while imperative programming is often more complex and requires more careful programming.
 - Imperative Programming – A programming paradigm that describes **how** a task should be performed **using step-by-step instructions**. Focuses on commands, **control flow**, and state changes; program **control is handled explicitly by the programmer**; usually more detailed and complex. Examples: **C** Programming Language, **Java**.
 - Declarative Programming – A programming paradigm that describes **what** result is required **without specifying the exact steps**. Focuses on **desired outcomes and logic**; program control is managed by the language or system; usually simpler and shorter. Examples: **SQL**, **Haskell**.
- Ultimately, the decision to use declarative or imperative programming **depends on the problem** you are trying to solve and your specific programming needs.

Imperative Programming Paradigm?

- Imperative Programming (Tells how to do something)
- You give step-by-step instructions for what the computer should do.
 - Example (in C):

```
/* C Program to Calculate Square of a Number */  
  
#include<stdio.h>  
  
int main()  
{  
    int number, Square;  
  
    printf(" \n Please Enter any integer Value : ");  
    scanf("%d", &number);  
  
    Square = number * number;  
  
    printf("\n Square of a given number %d is = %d", number, Square);  
  
    return 0;  
}
```

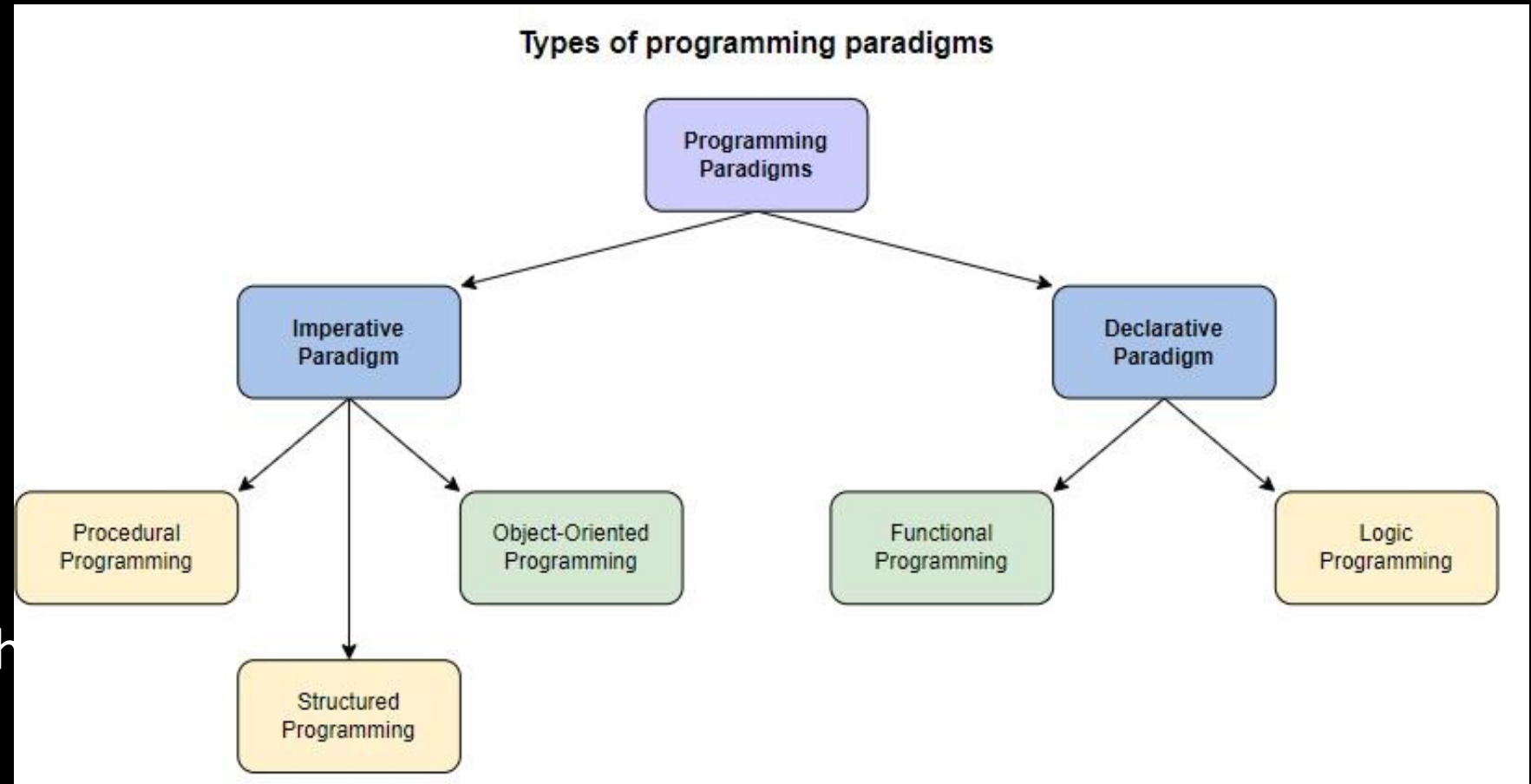
- This says how to square each number: loop through the list, multiply each, and store it.

Declarative Programming Paradigm?

- Declarative Programming (Tells what you want)
- You describe what you want, not how to do it. The language or system figures out the "how."
- Example (in SQL):
 - `SELECT name FROM students WHERE grade > 90;`
- This says *what* you want: the names of students with grades over 90. You do not write the steps to get them.

Types of Programming Languages

- imperative programming languages include:
- Java, C, Pascal
- declarative programming languages include SQL, Prolog
- Python supports both declarative and imperative programming



Main Programming Paradigms

1. **Imperative** Paradigm – Focuses on **how** a program executes step-by-step instructions. Examples: C Programming Language, Java.
2. **Procedural** Paradigm – Organizes programs into **procedures or functions** for modular programming. Examples: C Programming Language, Pascal.
3. **Object-Oriented** Paradigm – Uses objects and classes to model real-world entities. Examples: Java, C++, Python.
4. **Declarative** Paradigm – Focuses on what result is required rather than how to achieve it. Examples: SQL, Haskell.
5. **Functional** Paradigm – Uses **mathematical functions and immutable data**. Examples: Haskell, Lisp.
6. **Logic** Paradigm – Based on logical **rules and facts** for problem solving. Example: Prolog.
7. **Event-Driven** Paradigm – Program **flow is controlled by events** such as clicks or keyboard input. Example: JavaScript.

Computational Thinking ctd..

3. **Solution Implementation & Evaluation**: Finally, we create the actual solution and systematically evaluate it to determine its **correctness and efficiency**.

This step also involves seeing **if the solution can be generalized** via automation or extension to other kinds of problems.

Computational Thinking and Problem Solving

- When we use computational thinking to solve a problem, what we are really doing is **developing an algorithm: a step-by-step series of instructions**.
- Whether it is a **small task like scheduling meetings** or a large task like **mapping the planet**, the ability to develop and describe algorithms is crucial to the problem-solving process based on computational thinking.



- Solving Problems at Google Using Computational Thinking
- <https://www.youtube.com/watch?v=SVVB5RQfYxk>

Details of the Computational Thinking Approach ctd..

1. Problem Specification

- Computational Thinking *Principles*: Model Development with Abstraction, Decomposition, and Pattern Recognition
- Computer Science *Concepts*: Problem Analysis and Specification
- Software Engineering *Concepts*: Requirements Specifications Document, UML diagrams etc.

Details of the Computational Thinking Approach ctd..

2. Algorithmic Expression

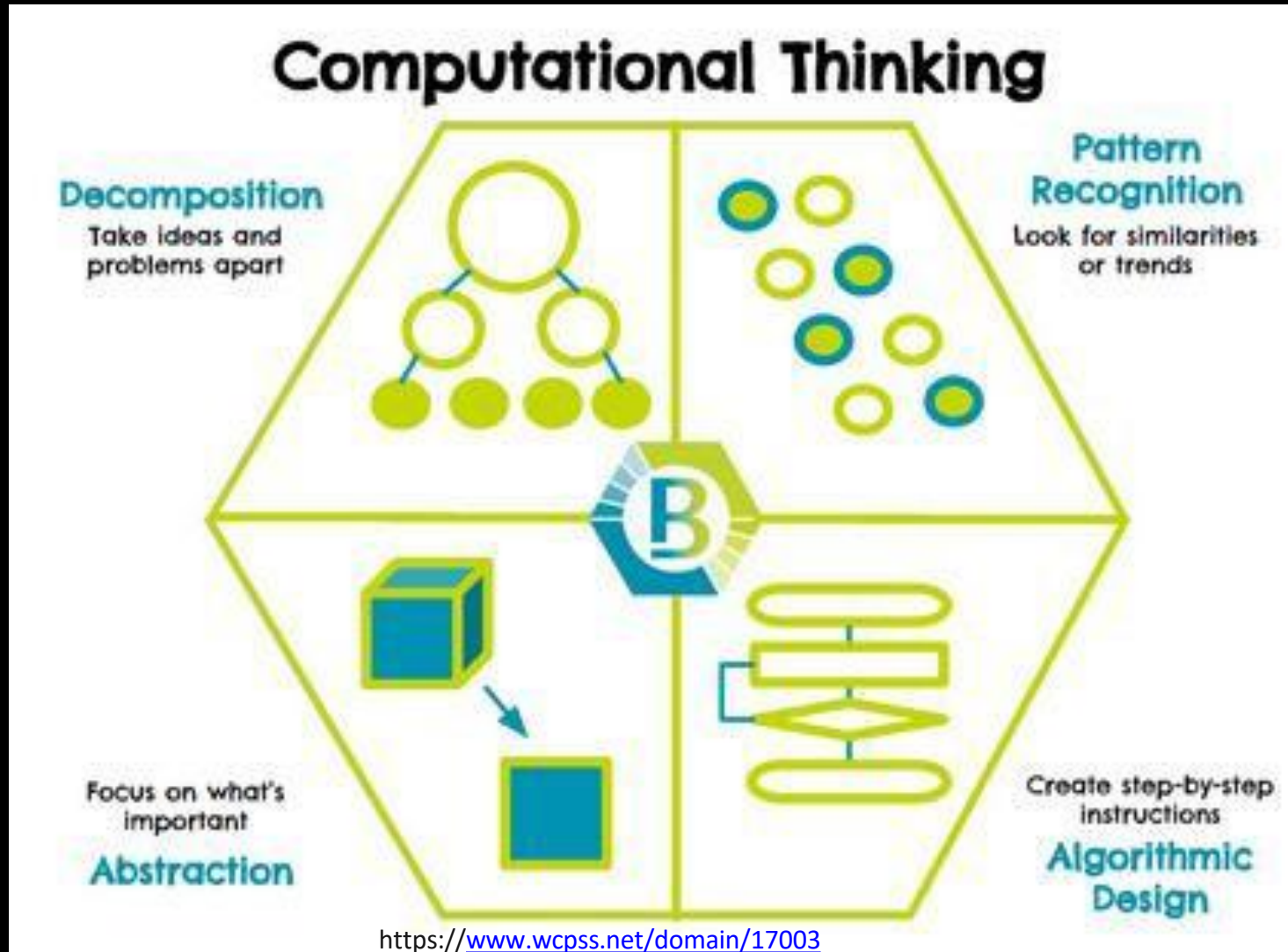
- Computational Thinking *Principles*: Computational Problem Solving using **Data Representation** and **Algorithmic Design**
- Computer Science *Concepts*: Data representation via some symbolic system and algorithmic development to systematically process information using **modularity, flow control** (including **sequential, selection, and iteration**), **recursion, encapsulation, and parallel computing**
- Software Engineering *Concepts* : Flowcharts, Pseudocode, Data Flow Diagrams, State Diagrams, Class-responsibility-collaboration (CRC) cards for Class Diagrams, Use Cases for Sequence Diagrams, etc.

Details of the Computational Thinking Approach ctd..

3. Solution Implementation & Evaluation

- Computational Thinking *Principles*: Systematic Testing and Generalization
- Computer Science *Concepts*: Algorithm implementation with analysis of efficiency and performance constraints, debugging, testing for error detection, evaluation metrics to measure correctness of solution, and extending the computational solution to other kinds of problems
- Software Engineering Concepts: Implementation in a Programming Language, Code Reviews, Refactoring, Test Suites using a tool like JUnit for Unit and System Testing, Quality Assurance (QA), etc.

Key Pillars of Computational Thinking

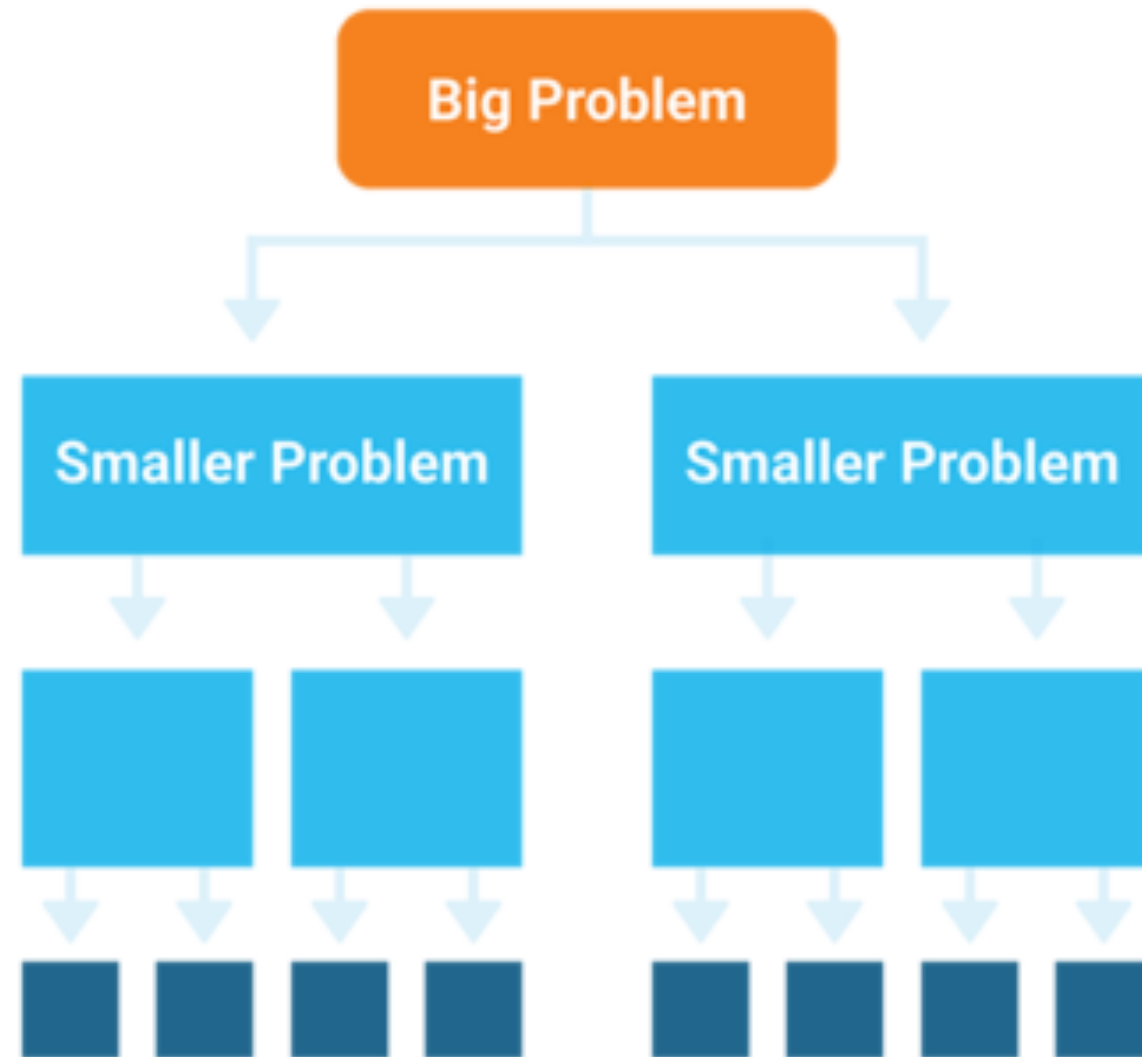


Computational Thinking

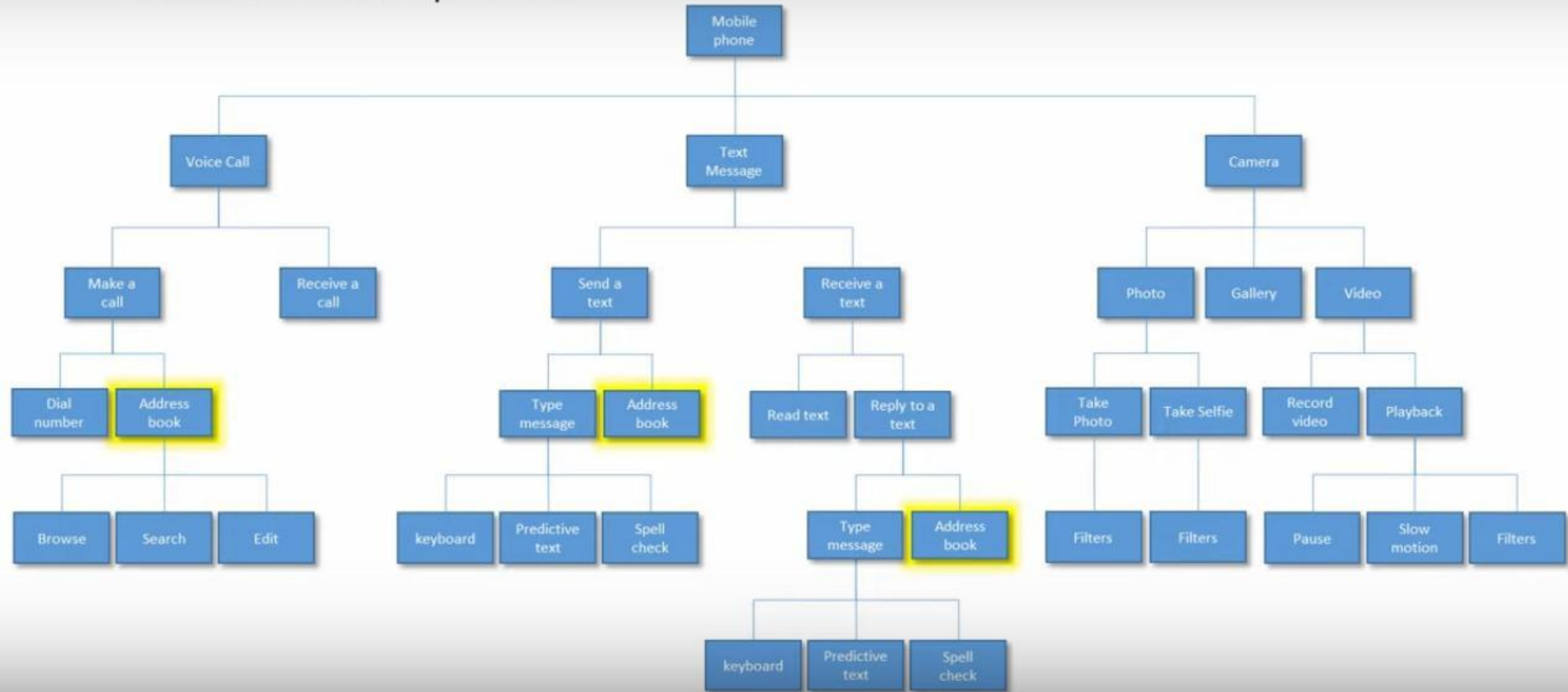
- Computational Thinking is an approach to problem solving with four key thinking processes;
 1. **decomposition**- taking ideas and problems apart, taking that complex problem and breaking it down into a **series of small, more manageable problems**
 2. **pattern recognition**- looking for **similarities or trends**, smaller problems can then be looked at individually, considering how similar problems have been solved previously
 3. **abstraction**- focusing on what is most important, focusing only on the important details, while ignoring irrelevant information and
 4. **algorithm design**- creating step-by-step instructions to solve a problem, simple steps or rules to solve each of the smaller problems can be designed.

Decomposition

- It is a problem-solving approach that allows individuals to tackle **complex tasks by dividing them into simpler subtasks**.
- Process allows individuals can **better understand complex problems** and find efficient solutions.
- Enables to **focus on** addressing each **component individually**, **easier to manage** and solve the overall problem.
- helps in identifying patterns and relationships among the smaller parts, leading to a deeper understanding of the problem as a whole.
- When faced with a complex problem, decomposition allows individuals to **prioritize and allocate their time effectively**.
- By dividing the problem into smaller parts, they can allocate **time** to address each subtask based on its **importance and urgency**



Functional Decomposition



Decomposition

- Another benefit of decomposition is the opportunity it **provides for delegation and collaboration**.
- Breaking down a complex problem into smaller parts enables **individuals to distribute the workload among a team**, improving efficiency and productivity.
- By breaking down complex problems into smaller, more manageable parts, individuals can develop **a deeper understanding of the problem** and approach it **more effectively**.
- Decomposition enhances **critical thinking, time management, delegation, and collaboration** skills, making it an essential skill for problem-solving in various domains

Decomposition

- The smaller parts can then be examined and solved, or designed, individually, as they are simpler to work with.
- If a problem is **NOT decomposed**, it is **much harder** to solve. Dealing with **a complex problem is much more difficult than breaking a problem** down into a number of smaller problems and solving each one, one at a time.
- **Smaller** problems are easier to understand and can be examined in more detail.
 - For example, suppose that a crime has been committed. Solving a crime can be a very complex problem as there are many things to consider.

Decomposition

- A police officer would need to know the **answer to a series of smaller problems**:
 - i. **What** crime was committed
 - ii. **When** the crime was committed
 - iii. **Where** the crime was committed
 - iv. **What** evidence there is
 - v. **If** there were any witnesses
 - vi. **If** there have recently been any similar crimes
- The complex problem of the committed crime has now been broken down into simpler problems that can be examined individually, in detail.
- Once the individual information has been gathered and collated, the police officer may be able to solve the crime.

Decomposition

- Question: How might a programmer decompose the complex problem of how to create an app?
- The problem might decompose into these simpler problems:
 - what kind of app they want to create
 - what the app will look like
 - who the target audience for the app is
 - what the graphics will look like
 - what audio will be included
 - what software they will use to build the app
 - how the user will navigate the app
 - how they will test the app
 - where they will sell the app
- These smaller problems, solved individually, will help the programmer create an app.

Examples of Decomposition in Everyday Life

- Decomposition is something we inherently do in our **daily lives, even if we do not realize it.**
- If you arrange a meal, you can use decomposition to select the menu, enlist support from others in the kitchen, task people with what to bring, determine the process by which to cook the different elements, and set the time for the meal

Everyday Examples of Decomposition

- When we taste an unfamiliar dish and identify several ingredients based on the flavor, we are decomposing that dish into its individual ingredients
- When we give someone directions to our house, we are *decomposing* the process of getting from one place to another (e.g., city, interstate, etc.)
- When we break a course project into several steps, we are decomposing the task into smaller, more manageable subtasks
- In mathematics, we can *decompose* a number such as **256.37** as follows: $2*10^2+5*10^1+6*10^0+3*10^{-1}+7*10^{-2}$

Examples of Decomposition in Computer Science

- From a computer science and coding perspective, decomposition can come into play programming a new game.
- For example, need to consider the characters, setting, and plot, as well as consider **how different actions will take place, how it will be deployed, and so much more**



Character Actions			
Standard	Movement	Attack	Special
Left	Walk	Spell	Mix Potion
Right	Jump	Confuse	Regenerate
Up	Crouch	Throw	
Down		Punch	

Pattern Recognition

- It involves the ability to **identify similarities and differences** in the details of a problem, allowing individuals to **simplify complex problems** by focusing on the **underlying patterns**.
- The ability to recognize patterns is vital because it helps individuals **break down** a problem into **smaller, more manageable parts**.
- By identifying similarities across different components of a problem, individuals can **apply a single solution to multiple instances**, saving time and effort.
- Similarly, recognizing differences between components helps individuals understand the **unique** aspects of each part and create **specific solutions** accordingly.

Pattern Recognition

- For example let us say we want to calculate the first 5 **square numbers**. This involves 5 sums:

- $1*1$
- $2*2$
- $3*3$
- $4*4$
- $5*5$
- For each number we are doing the same thing: multiplying it by itself.
- We can easily take advantage of that fact to write a function, or use a for loop or similar construct.

- Sometimes it is harder to define a pattern.
For example let us say we want to calculate the first 5 **triangular numbers**. This involves 5 sums:

1

1+2

1+2+3

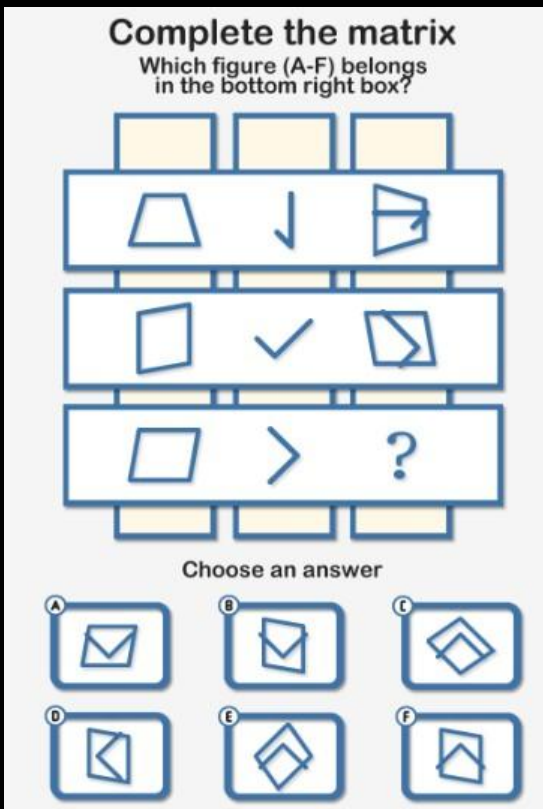
1+2+3+4

1+2+3+4+5

We can see a pattern here but it is not **as simple as the square number example**.

It is a **pattern of two stages**: first we need to create a list or sequence of the numbers, before we can sum them.

Now that we have identified the pattern, we can try to construct some code to automate/simplify its implementation.



Pattern Recognition

- All about recognizing patterns. Specifically, with computational thinking, **pattern recognition occurs as different decomposed problems are studied.**
- Through analysis, it is possible to recognize **patterns or connections** among the **different pieces** of the **larger problem.**
- These patterns can be **both shared similarities** and **shared differences.**
- This concept is essential to building understanding complex information and goes well beyond recognizing patterns amongst sequences of numbers, characters, or symbols

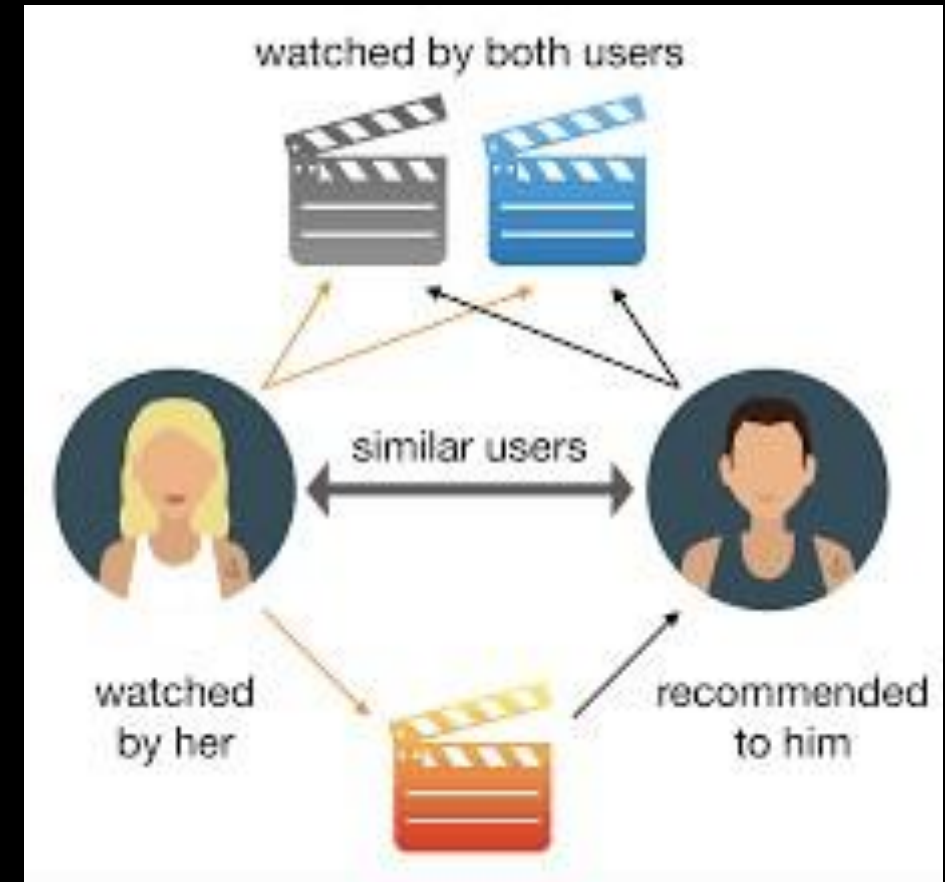
Examples of Pattern Recognition in Everyday Life

- Pattern recognition is the foundation of our knowledge.
- As infants, we used patterns to make sense of the world around us, to begin to respond verbally and grow our language skills, to develop behavioral responses, and to cultivate connections in this world.
- Beyond this, pattern recognition also occurs when scientists try to identify the cause of a disease outbreak by looking for similarities in the different cases to determine the source of the outbreak
- Question: Pattern Recognition in Medical and Financial Domains?



Examples of Pattern Recognition in Everyday Life

- When Netflix recommends shows based on your interests or a chat bot annoys you on a website, the technology (e.g Artificial Intelligence and Machine Learning) relies on pattern recognition.
- Movie Recommendation Systems, Clickstream Analysis on browser behavior
- Animals can be classified based on their Characteristics



https://rpubs.com/ezrasote/movie_recommendation

CODE REUSE

DEVELOPER NO.1

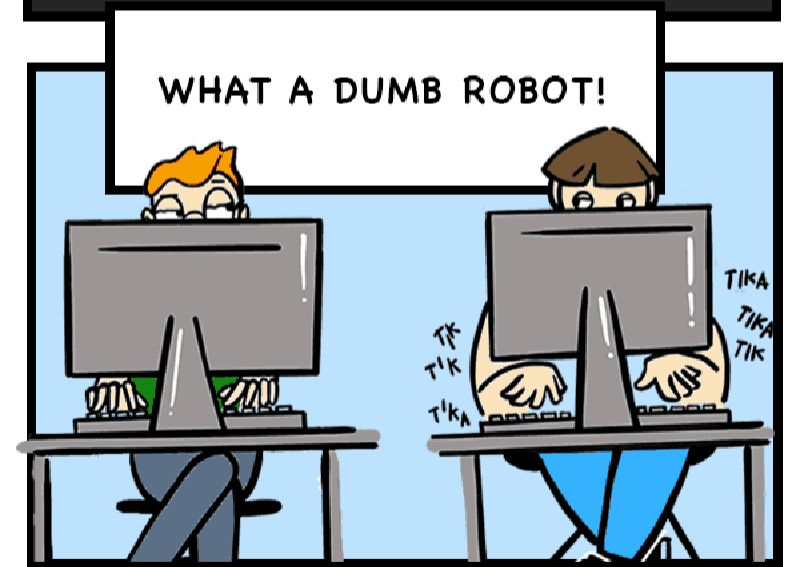
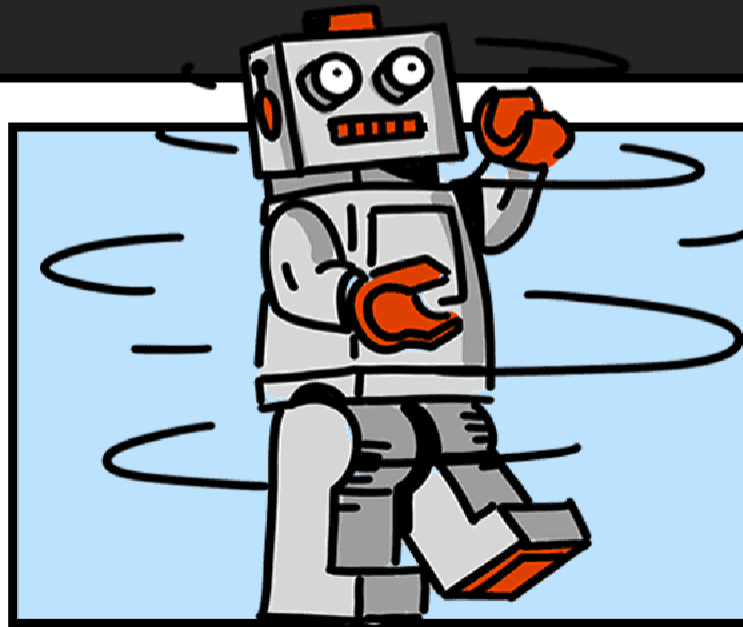
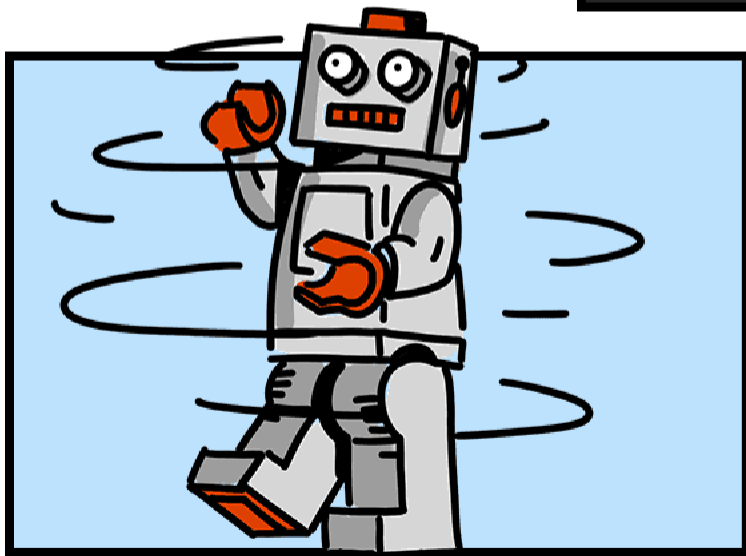
I'LL CREATE FUCTIONALITY FOR TURNING LEFT, CAN YOU DO IT FOR TURNING RIGHT?

```
int turn_left() {  
    float desired_o = get_orientation() + M_PI / 2;  
    right_motor_pwm(100);  
    while(orientation() < desired_o) { noop(); }  
    right_motor_pwm(0);  
    return 0;  
}
```

DEVELOPER NO.2 TYPING CODE...

```
int turn_right() {  
    for(int i = 0; i < 3; i++) {  
        turn_left();  
    }  
    return 0;  
}
```

SURE THING.



spinning around it's axis

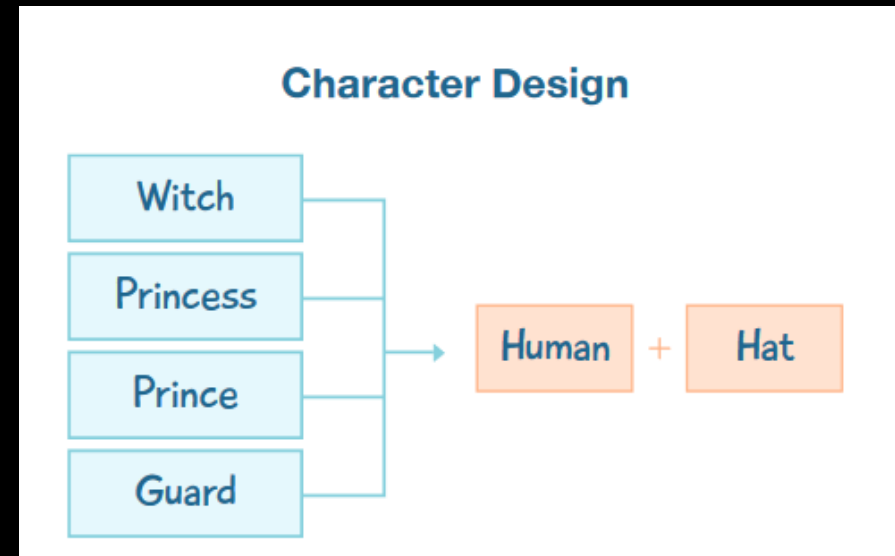
TEXT: STEFAN VUKANIC', SPFR

ILLUSTRATION: DRAGANA KRGINIC', SPFR

SPICY COMICS

Examples of Pattern Recognition in Computer Science

- In computer science, **pattern recognition** helps students **identify similarities between decomposed problems**. If they are coding a **game**, they may recognize **similar objects, patterns, and actions**.
- Finding these allows them to apply the **same, or slightly modified, string of code to each**, which makes their programming more efficient



Examples of Pattern Recognition

- Drivers look for patterns in traffic to **decide whether and when to switch lanes**
- People look for patterns in **stock prices** to decide when to buy and sell
- Scientists look for patterns in data to **derive theories and models**
- We look for patterns and learn from them to avoid repeating the same mistake

example scenario in computing:

(a) Certain classes of ML algorithms focus on looking for hidden patterns or clusters of features in the data

(b) Reinforcement

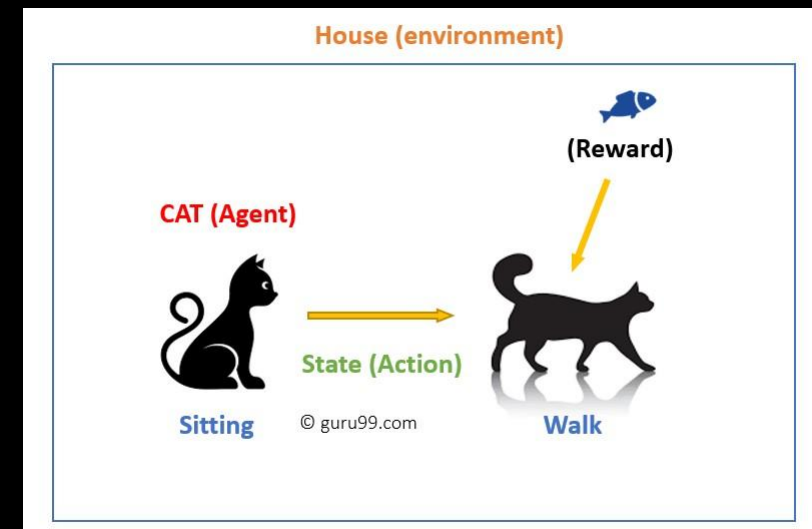
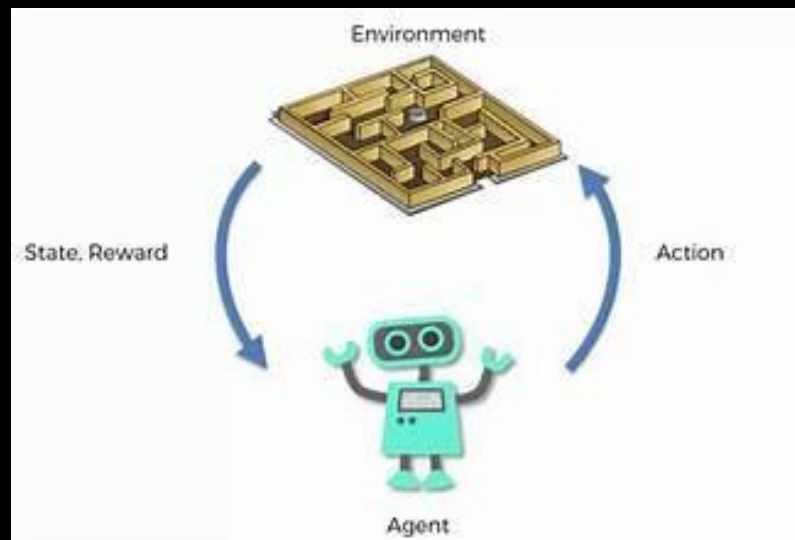
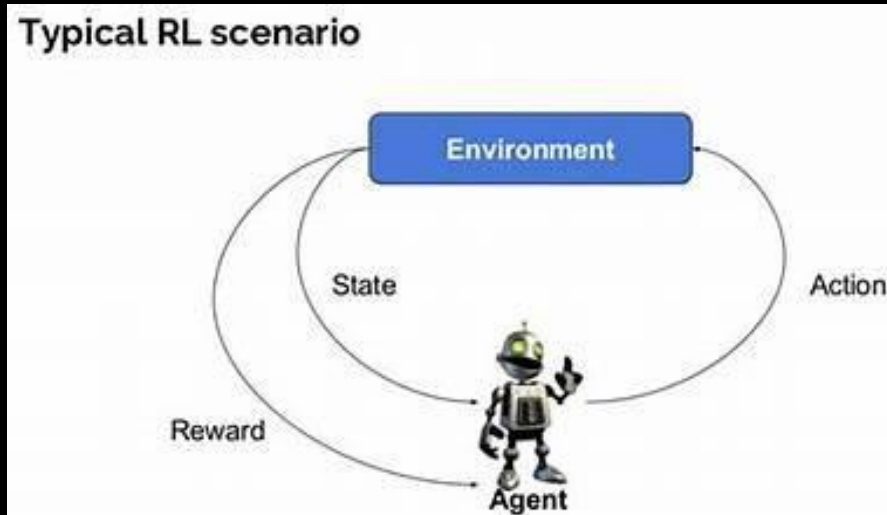
Learning(Reward and Punishments)

(**to be leaned in later modules in degree program)

Example: Deep Reinforcement Learning for Autonomous

https://www.media.mit.edu/projects/drl_a UAV/overview/

Reinforcement Learning Uses Patterns



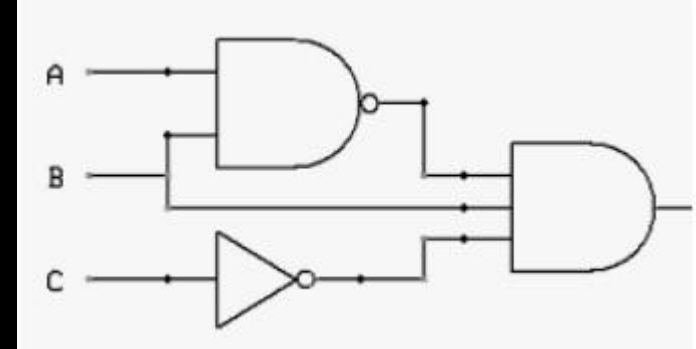
Examples of Pattern Recognition

<u>Problem Domain</u>	<u>Application</u>	<u>Input Pattern</u>	<u>Pattern Class</u>
Bioinformatics	Sequence Analysis	DNA/Protein sequence	Known type of genes/patterns
Data mining	Searching for meaningful patterns	Points in multi dimension space	Compact and well separated clusters
Document classification	Internet search	Text document	Semantic categories
Document image analysis	Reading machine for the blind	Document image	Alphanumeric characters / words
Industrial automation	Printed circuit board inspection	Intensity or range image	Defective / non defective nature of product
Multimedia database retrieval	Internet search	Video clip	Video genres e.g. action, dialogue etc
Biometric recognition	Personal identification	Face, iris & finger print	Authorized user for access control
Remote sensing	Forecasting crop yield	Multispectral image	Land use categories, growth pattern of crops
Speech recognition	Telephone directory enquiry with operator	Speech waveform	Spoken words

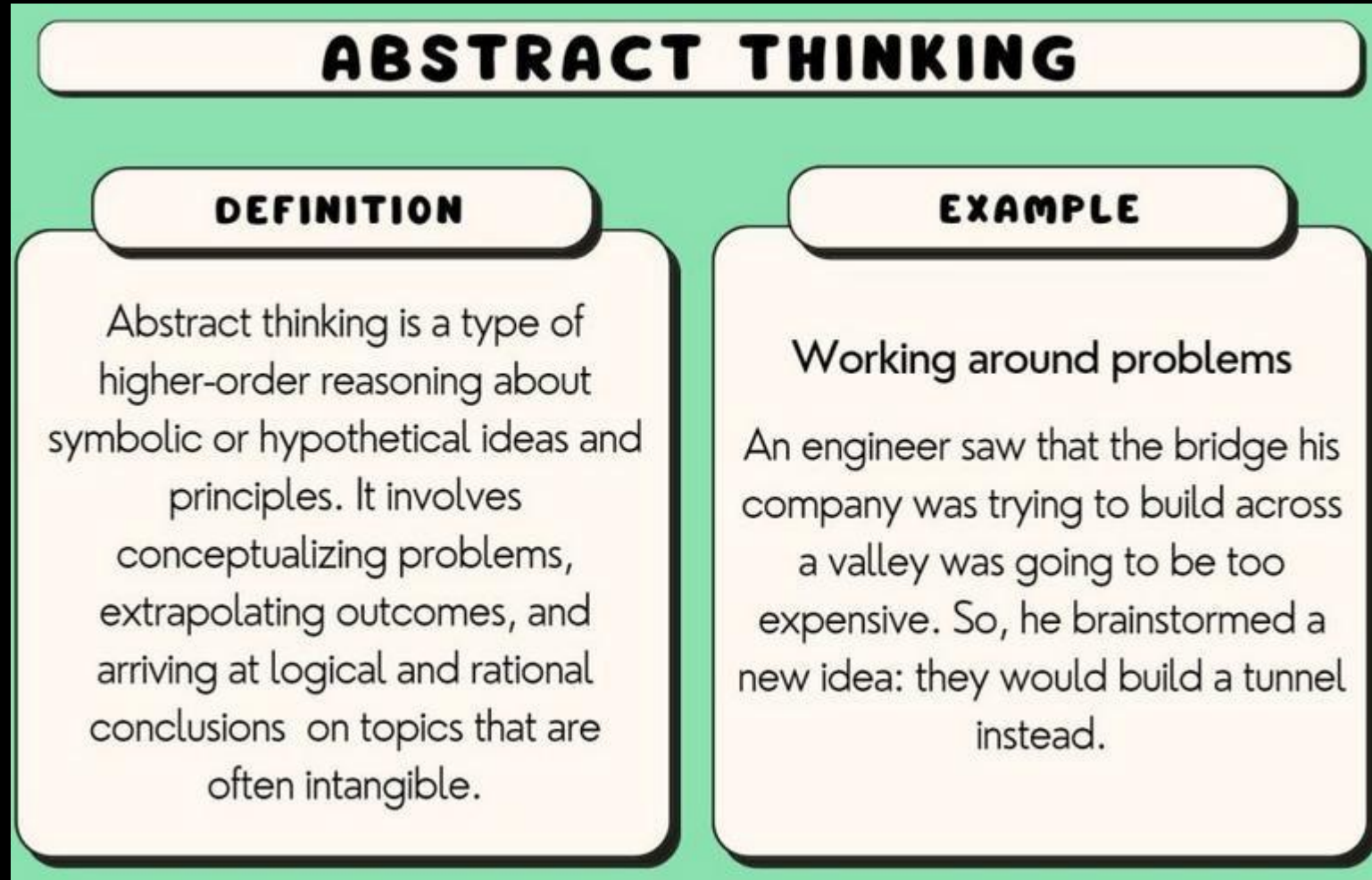
Abstraction

- Abstraction allows us to **create a general idea** of what **the problem is and how** to solve it.
- The process instructs us to **remove all specific detail** and **any patterns that will NOT help** us solve our problem.
- This helps us to form our idea of the problem.
- It allows individuals to focus on the **essential aspects** of a problem and **disregard irrelevant details** that may distract from finding a solution.

Examples of Abstraction



Abstract Thinking Example

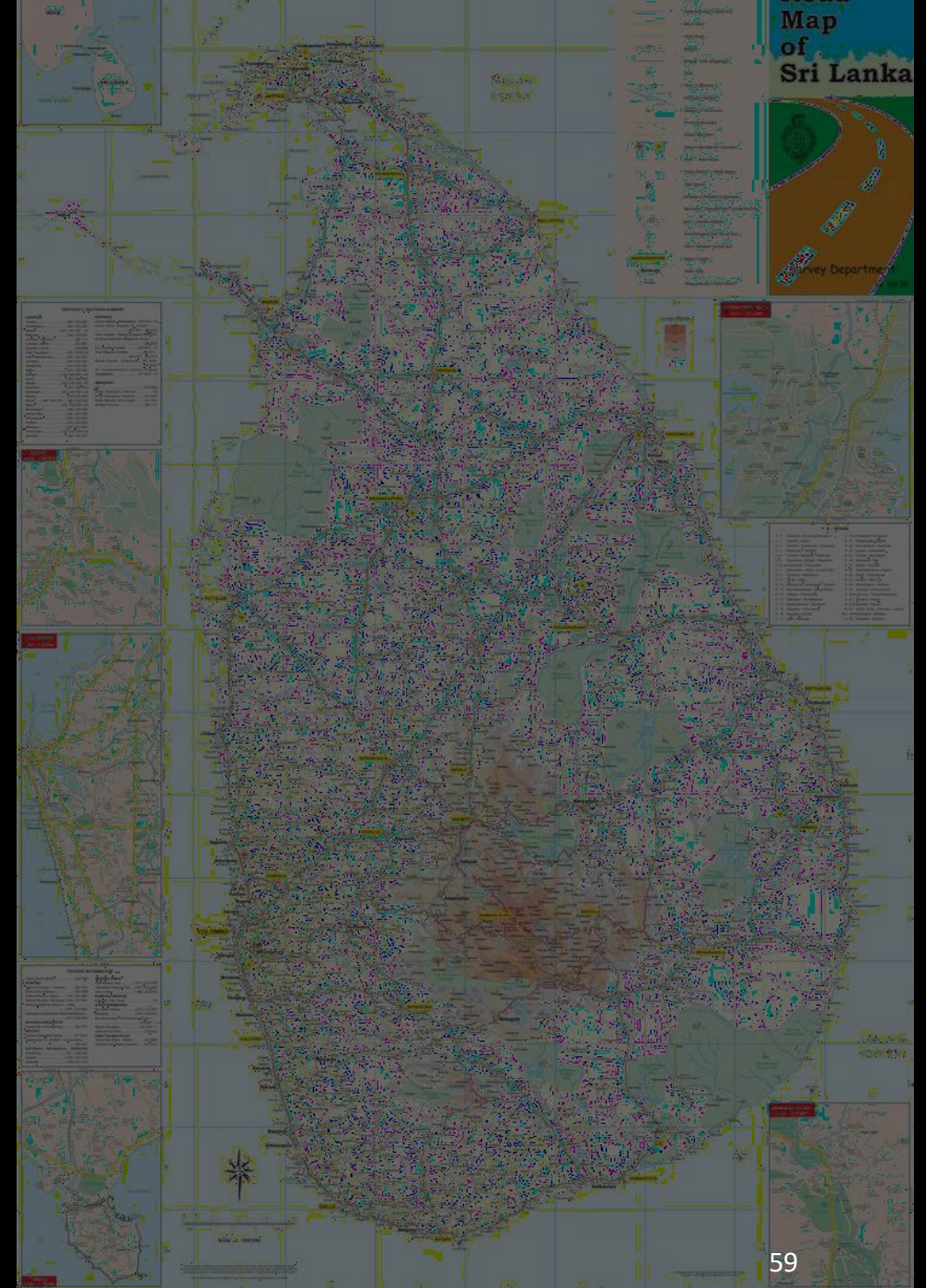


Example

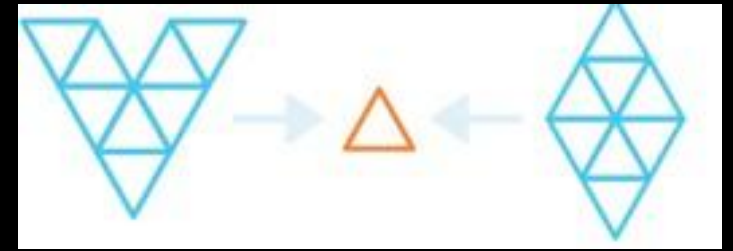
- Consider the problem of creating a program to **calculate the area of shapes**. The **problem** could first **be decomposed into modules**, each of which would be a particular shape, for example **rectangle, square and triangle**. **Abstraction** can then be followed for each module.
- For example, for the rectangle module the first step would be to notice that all rectangles share general characteristics:
 - a width
 - a height
 - $\text{area} = \text{width} \times \text{height}$
- When **abstracting, certain details are discarded** but **others are kept**:
 - all rectangles have a width, but for the **program design** the **actual** rectangle width is **NOT needed**
 - all rectangles have a height, but for the program design the actual rectangle height is not needed
 - area is always $\text{width} \times \text{height}$
- To solve this problem, all the program needs to be able to do is **receive a width and a height, then calculate the area** from those numbers. The actual numbers are irrelevant - they change with every rectangle - so they are discarded.

Abstraction

- An example of abstraction is the use of a road map. It details roads (Types A, B etc), Cities, rail tracks etc .
- Those information are sufficient for a person to plan a journey .
- Other details, such as real geographical location, width of roads, whether carpeted road etc are not included as they are irrelevant to journey planning



Abstraction



- **Some algorithms** focus on looking for **hidden patterns or clusters** of features in the data called, **pattern generalization**, abstraction enables us to navigate complexity and find relevance and clarity at scale.
- **Decomposition** and **pattern recognition broke down the complex**, and **abstraction** figures out how to work with **the different parts** efficiently and accurately.
- This **process** occurs through **filtering** out the **extraneous and irrelevant** in order to identify what's most important and connect each decomposed problem.
- **Abstraction** is **similar to the selective filtering** function in our **brains** that gates the neural signals with which we are constantly bombarded so we can make sense of our world and focus on what is essential to us.

Examples of Abstraction in Everyday Life

- **Another way to think about abstraction** is in the context of those large/complex concepts that inform how we think about the world like **Newton's Laws of Motion**, the Law of **Supply and Demand**, or the **Pythagorean Theorem**.
- All of these required the **people behind them**
 - to think about large, broad, and complex concepts;
 - to break down the problem and to experiment;
 - to find patterns amongst the experimentations; and
 - to eventually **abstract** this **concrete knowledge** to package it into these statements that free us from the **complexity and difficulty** went through to arrive at this law.

Computing & Abstraction

- Computing mostly operates **independently** of the **concrete world**.
- The **hardware implements a model of computation** that is interchangeable with others.
- The **software** is structured in **architectures** to enable humans to create the large/complex systems by **concentrating on a few issues at a time**.
- These **architectures** are made of specific choices of abstractions.

Abstraction



Car User: Thinking about the necessary features and functionalities of a car

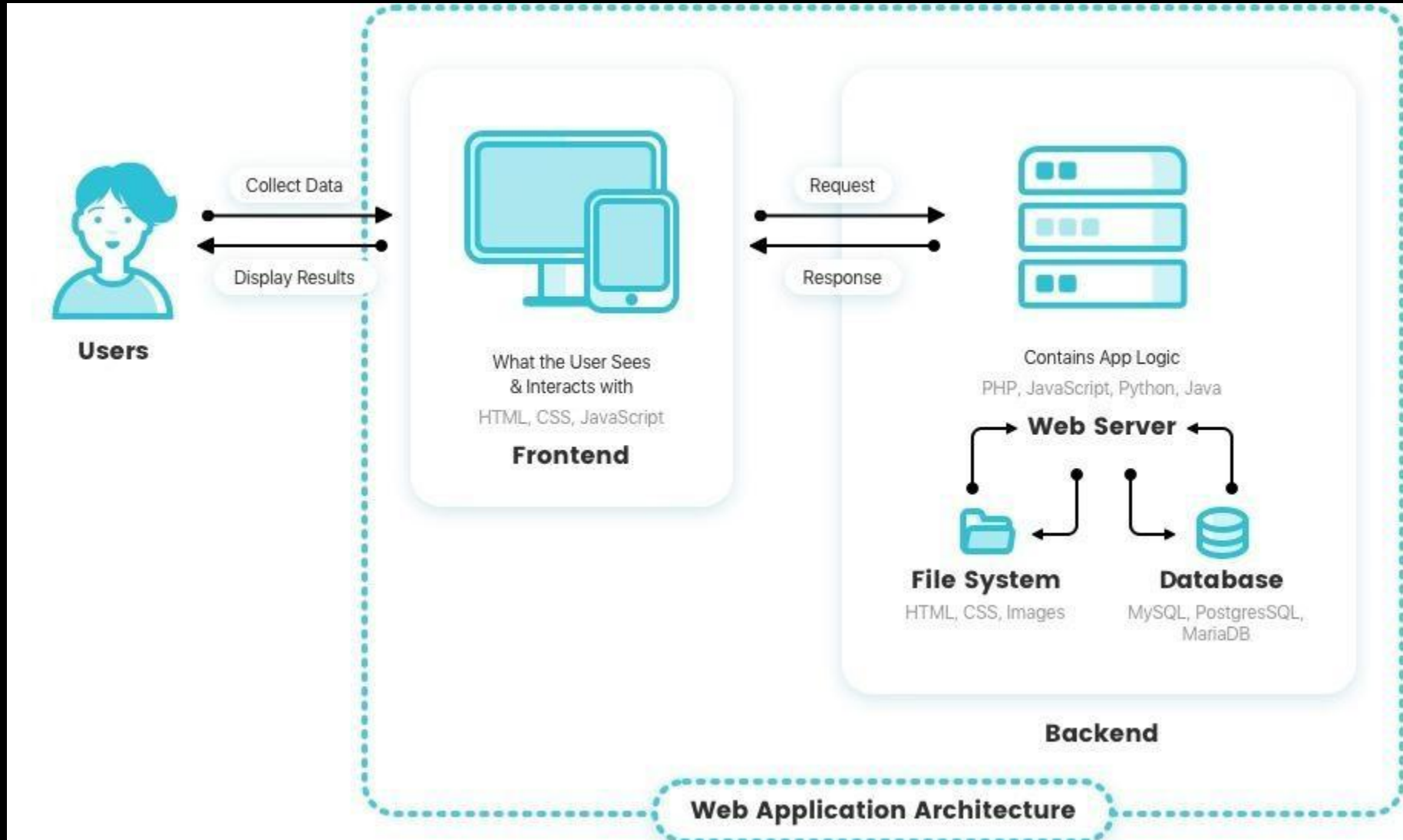


enjoyalgorithms.com

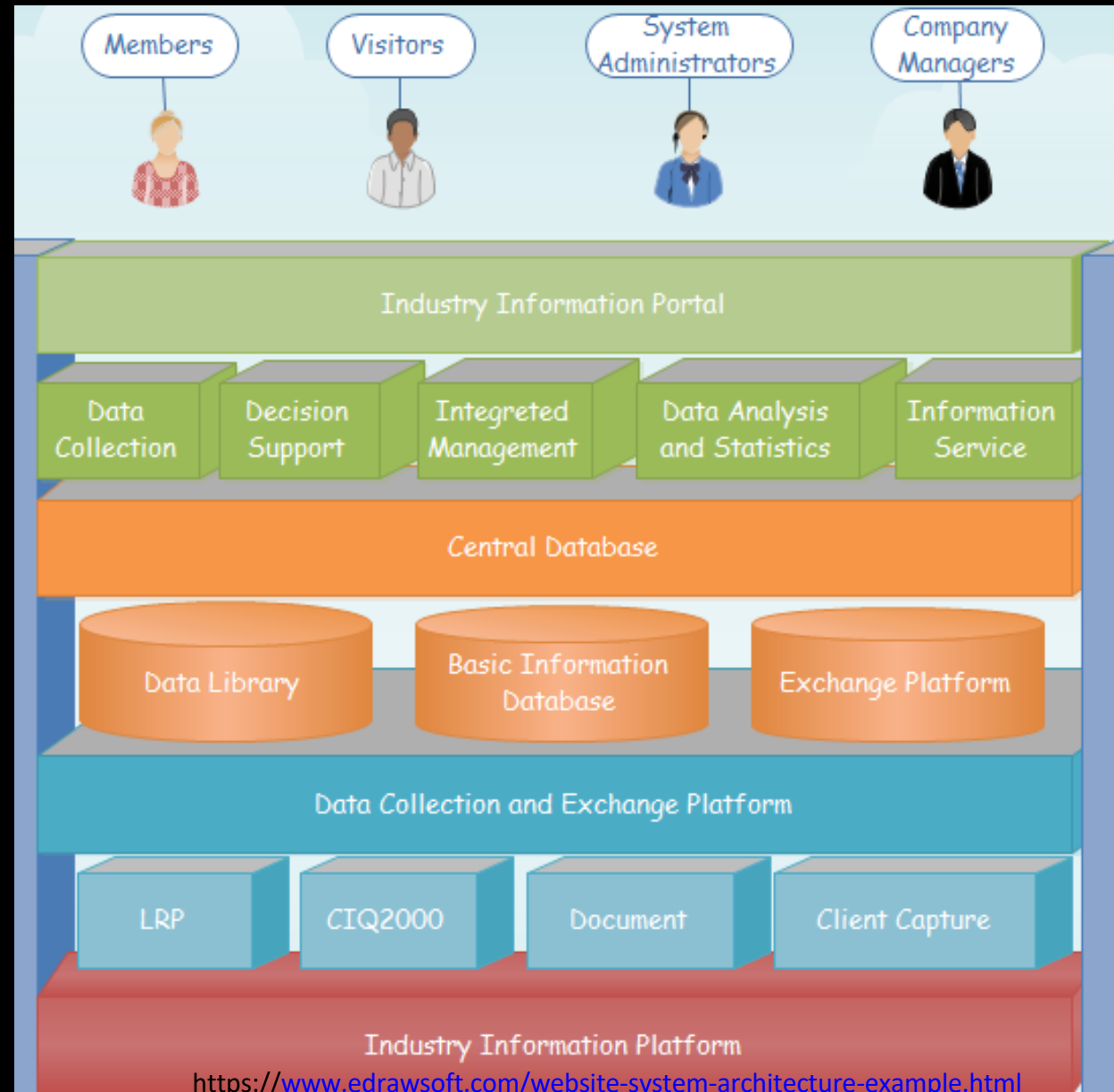


Automobile Engineer:
Thinking about the internal implementation of features and functionalities of a car

Example Web Application Architecture

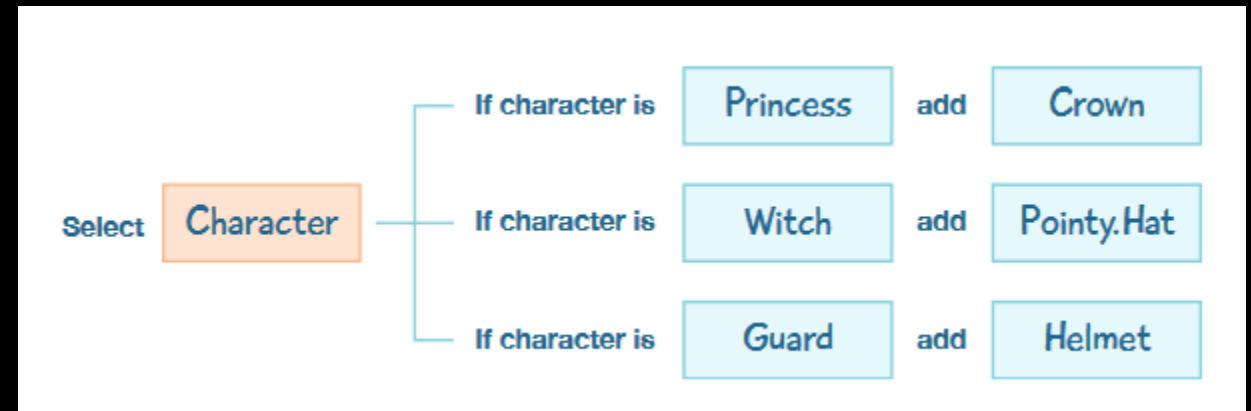


Website System Architecture Example



Examples of Abstractions in Computer Science

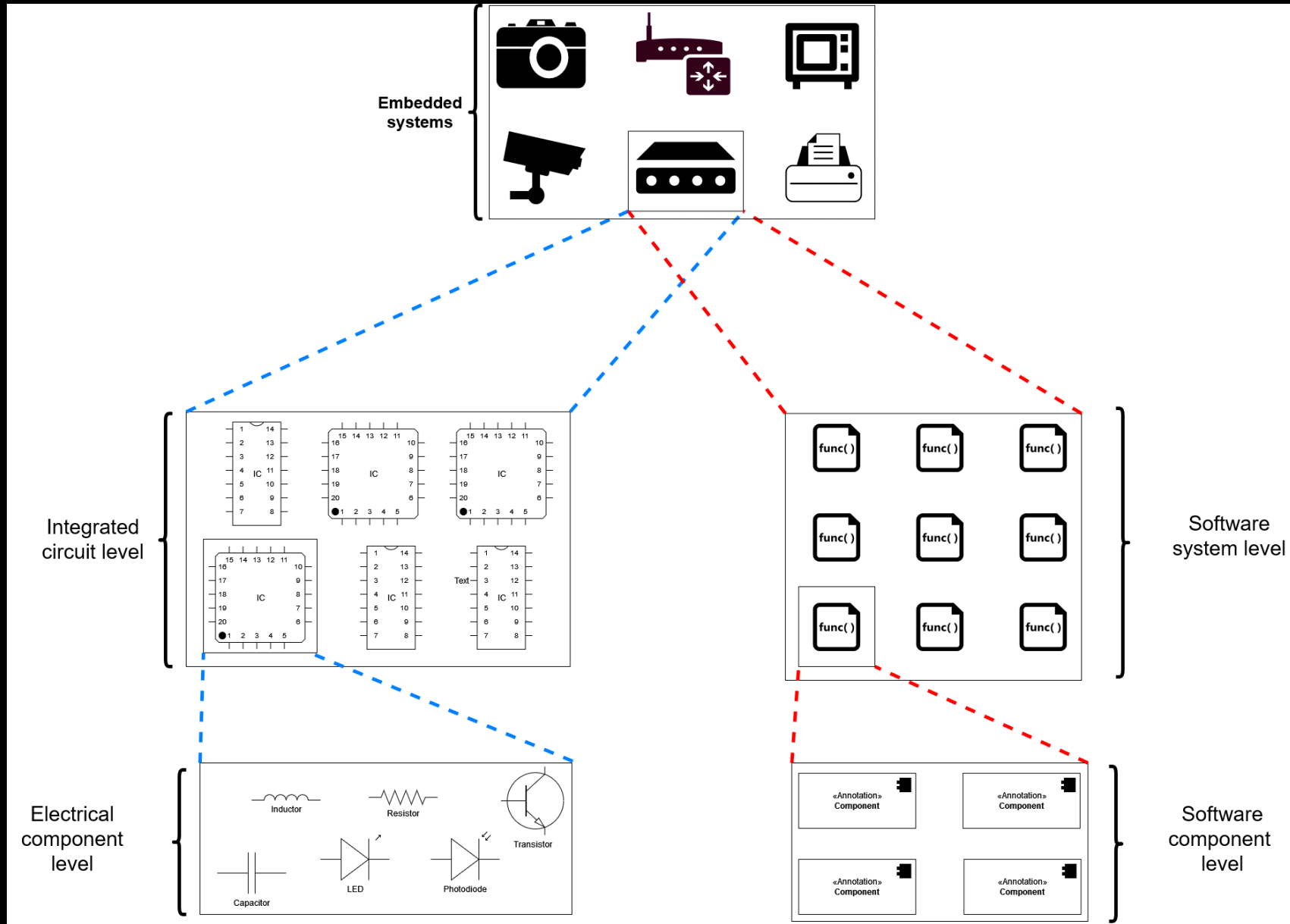
- Abstraction in coding (writing programs) is used to **simplify strings of code into different functions.**
- It **hides** the underlying **complexity** in a programming language, which makes it **simpler to implement algorithms and communicate with digital tools.**



Importance of Abstraction in Computing

- Understanding abstraction enables one to **make sense of problems** they encounter, helping them to **not be** surprised and confused in the face of **something complex and to persist, compute, iterate, and imagine**

Example of different levels of abstraction



Question



<https://www.goodfon.com/rendering/wallpaper-art-avtomobili-avtodrom.html>

- Imagine that a computer game is being designed to simulate cars on a race track. **Abstraction** has been used in the design.
- Explain how abstraction may be applied in the creation of the game.
 - The **controls** would most likely be overly **simplified** (e.g. accelerator, brake, and reverse simplified to forwards and backwards, gears likely removed)
 - The 3D models would likely be simplified polygonal models
 - The cars would likely not need re-fuelling – perhaps instead they have infinite fuel

Algorithmic Thinking

- An **algorithm** is a set of instructions that helps solve a specific problem or accomplish a particular task. These instructions are typically presented in a **clear and unambiguous manner**, allowing individuals or computers to **follow them precisely**.
- **Algorithmic Thinking** is a fundamental concept within Computational Thinking that involves **defining a step-by-step solution to a problem that can be replicated for a predictable outcome**, whether by humans or computers.
- ** It is the **process of breaking down a complex task into smaller, manageable steps and organizing them in a logical sequence**.
- In Algorithmic Thinking, **emphasis** is placed on the **design and structure of algorithms**.

Algorithmic Thinking

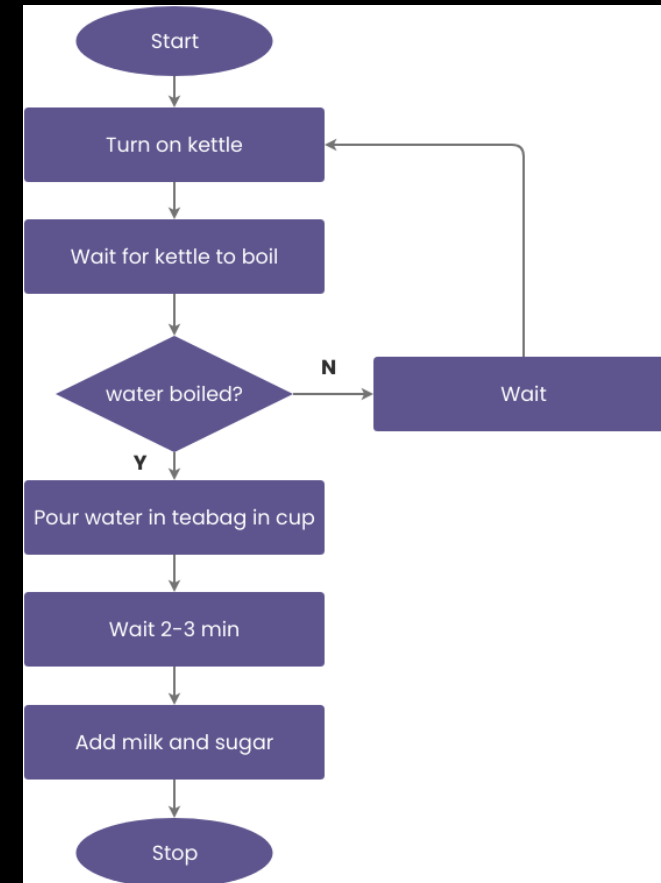
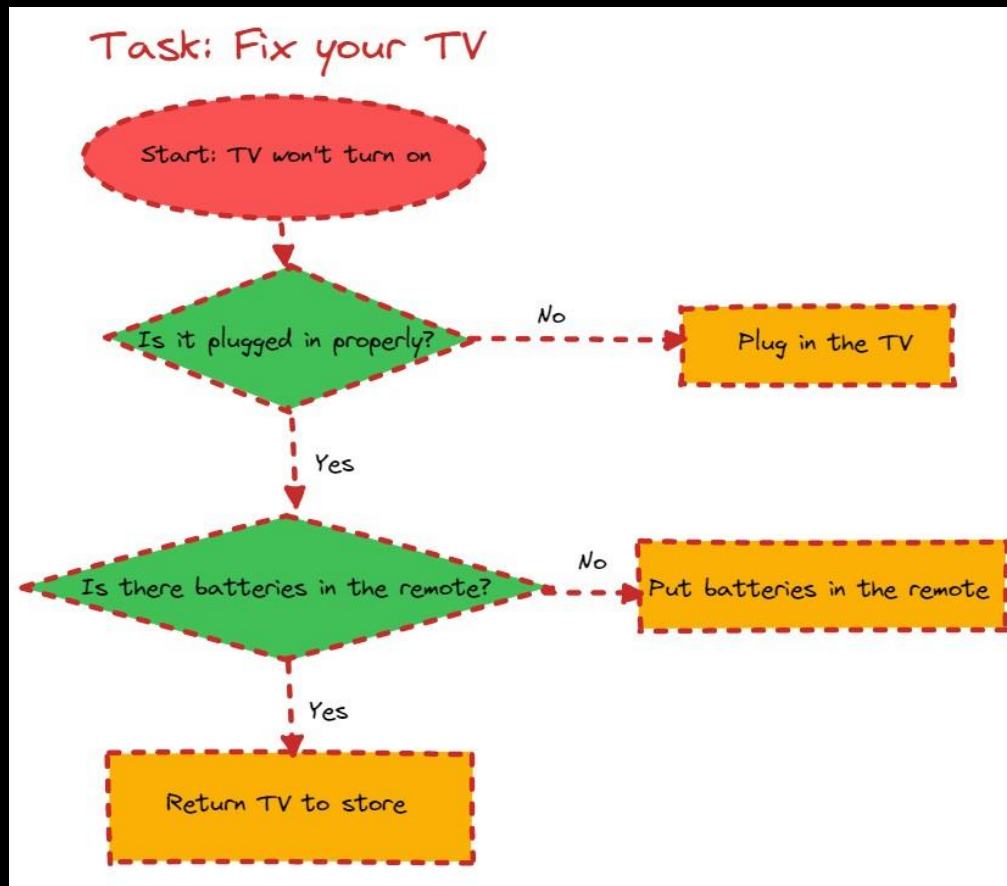
- Algorithmic thinking is a **derivative of computer science and the process to develop code and build applications.**
- This approach **helps the problem-solving process** by creating a series of **systematic, logical** steps that accept a defined set of **inputs** and **produce** a defined set of **outputs** based on these.
- In other words, algorithmic thinking is **not solving for a specific answer**; instead, it solves how to build a sequential, complete, and **replicable** process that has an end point – an algorithm. e.g Sorting, Merging, Searching(Google Search Algo)
- **Designing an algorithm** helps to **both communicate and interpret clear instructions** for a **predictable, reliable output.**

Algorithmic Thinking

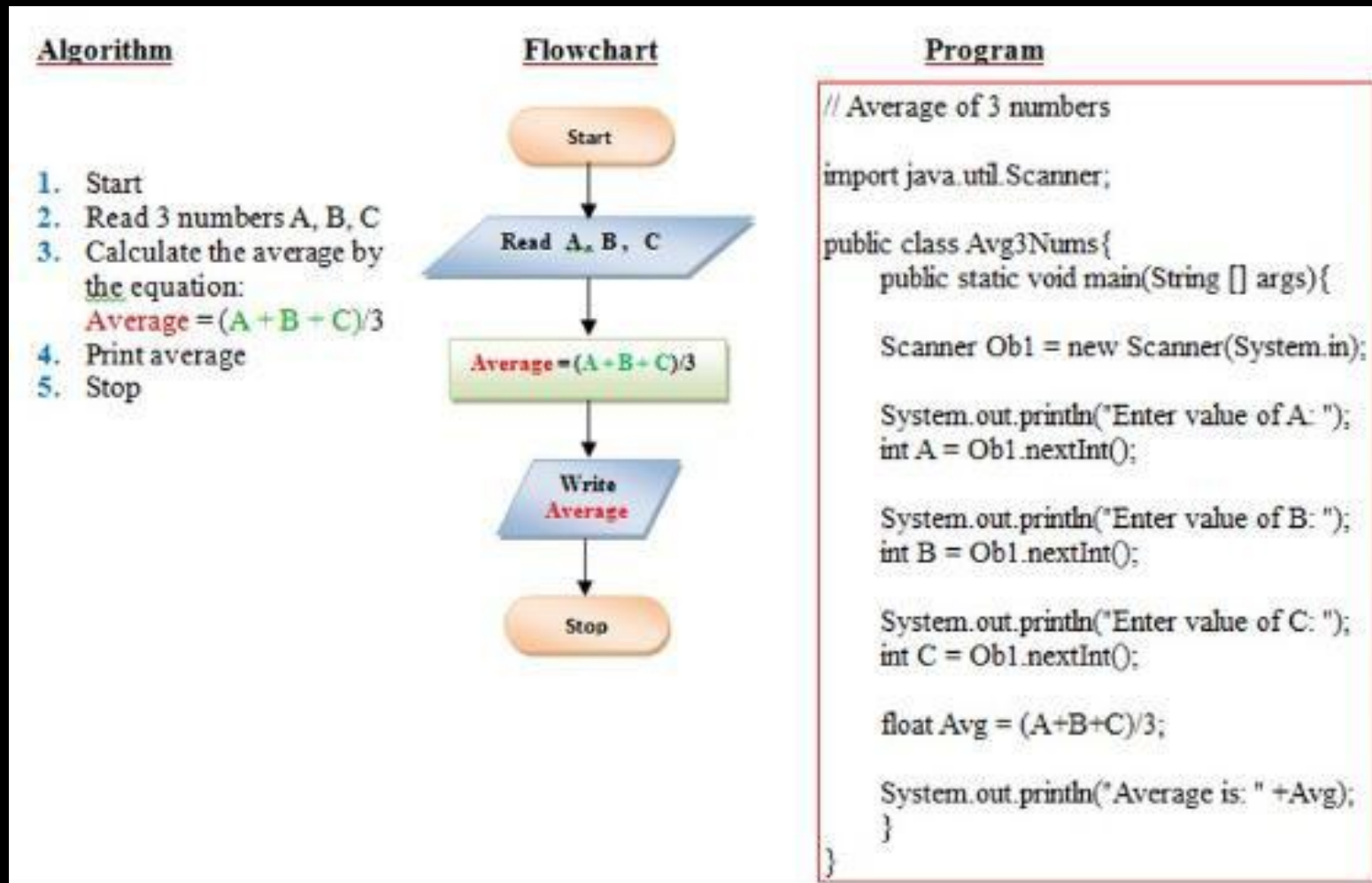
- The ability to think algorithmically is **vital** in the **problem-solving process**.
- It enables individuals to **approach challenges systematically** and **methodically**.
- By breaking down a problem into **smaller steps, identifying patterns, and identifying the appropriate sequence of actions**, algorithmic thinking helps to **simplify complex problems**.
- This structured approach enhances **efficiency, accuracy, and effectiveness in finding solutions**.
- Algorithm design is crucial in ensuring that the steps of the solution are well-defined, comprehensive, and optimized.
- A properly designed algorithm accounts for various **scenarios, considering potential errors or exceptions and providing contingency plans**.
- This systematic approach to algorithm design guarantees a more reliable and robust problem-solving process.

Examples of Algorithms in Everyday Life

- Similar to Computational thinking and its other elements discussed, **algorithms** are something we experience quite regularly in our lives.



Examples of Algorithms in Computations



Examples of Algorithms in Computer Science

- Algorithms used in coding are often complex.
- To contextualize algorithms in computer science and programming, following are a few examples.

Google and Algorithms



- How does search work in general ?
- The search process generally takes place in three stages:
 - Crawling. The search engine's algorithm directs web crawlers to discover URLs on the internet and examine their content. A crawler is a program that runs through content and automatically indexes it.
 - Indexing. The content contained in URLs is tagged with attributes and metadata(i.e data that provides information about other data) that help the search engine categorize the content.
 - Searching and ranking. The user enters a query, and the search engine ranks and returns content in relation to the query.
- *“Google's algorithms are complex mechanisms used to retrieve information from its search index and present the information to a given query. Algorithms examine billions of pieces of content in Google's index, looking for phrases and keywords that match the query.”*

Ref: <http://www.techtarget.com/whatis/feature/Google-algorithms-explained-Everything-you-need-to-know>

Examples Computational Thinking

1. Automating Repetitive Tasks: A data analyst at a tech company used computational thinking to automate a repetitive task of cleaning and organizing large datasets. By breaking down the task into simple steps and writing a script in a programming language, the analyst was able to save hours of manual work each week.
2. Optimizing Resource Allocation: A logistics manager at a shipping company used computational thinking to optimize the allocation of trucks for deliveries. By abstracting the problem and using computational tools, the manager was able to find the most efficient routes, reducing fuel costs and delivery times.
3. Improving Customer Service: A customer service manager at a retail company used computational thinking to improve the company's response time to customer inquiries. By analyzing patterns in customer complaints and creating an algorithm to prioritize responses, the company was able to improve its customer satisfaction ratings.
4. Enhancing Product Design: A product designer at a software company used computational thinking to enhance the design of a new app. By using logical reasoning to understand user needs and preferences, the designer was able to create a more user-friendly interface.
5. Predicting Market Trends: A financial analyst at an investment firm used computational thinking to predict market trends. By using computational tools to analyze historical data and identify patterns, the analyst was able to make more accurate predictions about future market movements.

How to Enhance your Algorithmic Thinking skills

- Explore Different Programming Paradigms(i.e Models/approaches)
- Expanding your knowledge of different programming paradigms can significantly enhance your algorithmic thinking by providing different perspectives on problem-solving.
- Paradigms to Explore:
 - Functional Programming: Learn languages like Haskell to understand immutability and pure functions.
 - Logic Programming: Try languages such as Prolog to experience declarative problem-solving.
 - Object-Oriented Programming: Learn and master deep into OOP principles Java, C++ etc.
- Each paradigm offers unique approaches to breaking down and solving problems, broadening your algorithmic toolkit.

Develop Abstract Thinking Skills

- Tackle Mathematical and Logical Puzzles
- Mathematical and logical puzzles are excellent for developing the abstract thinking skills crucial for algorithmic problem-solving.
- Types of Puzzles:
 - Sudoku and its variants
 - Logic grid puzzles
 - Mathematical word problems
 - Chess puzzles
- These puzzles train your brain to recognize patterns, think several steps ahead, and approach problems from multiple angles - all essential components of algorithmic thinking.

Develop Abstract Thinking Skills

- Key Concepts to Master:
 - Big O notation
 - Time complexity analysis
 - Space complexity analysis
 - Best, worst, and average case scenarios
- Will be covered in upcoming lectures

How to Enhance your Algorithmic Thinking skills

- Practice Pseudocode and Flowcharting
- Algorithmic thinking is not about coding in a specific language; it is about logical problem-solving. Pseudocode and flowcharts are excellent tools for developing this skill.
- Benefits:
 - Focuses on logic rather than syntax
 - Improves ability to communicate ideas
 - Helps visualize problem-solving steps
- Exercise:
 - Take everyday tasks (like making a tea or travelling to a city/building) and break down tasks into detailed pseudocode or flowcharts. This practice trains your brain to think in logical, step-by-step processes.

How to Enhance your Algorithmic Thinking skills

- Engage in Competitive Programming (Beyond LeetCode)
- Explore and use platforms for competitive programming that can enhance your algorithmic thinking.
- Explore:
 - [Leet Code](#): provides coding and algorithmic problems for users to prepare for technical interviews and coding competitions.
 - [HackerRank](#): Offers a wide range of programming challenges across various domains.
 - [CodeForces](#): Known for its regular coding competitions and challenging problems.
 - [TopCoder](#): Provides algorithmic challenges and hosts competitions with real-world applications.
- These platforms often provide diverse problem sets and can offer a fresh perspective on algorithmic challenges.

Questions?



Problem Solving Strategies and Computational Approaches SCS1304

Handout 2 : Introduction to Computational Thinking –
Part 2

Prasad Wimalaratne, PhD

Lecture Overview

- Introduction to Problem Solving Strategy
 - Linear Search
 - Binary Search
 - Fibonacci Sequence
 - Introduction to Recursion
 - Types of Recursion
 - Applications of Recursion
 - Best Practices and Pitfalls in using Recursion in Algorithmic Design
 - Recursion vs Loops/Iteration

Algorithms : Efficiency, Analysis, And
Order [Ref Chapter 1 of recommended book]

Goals of up coming lectures ..

- Design

- Problem solving method
- 5 design methods: divide-and-conquer, dynamic programming, greedy approach, backtracking, branch-and-bound

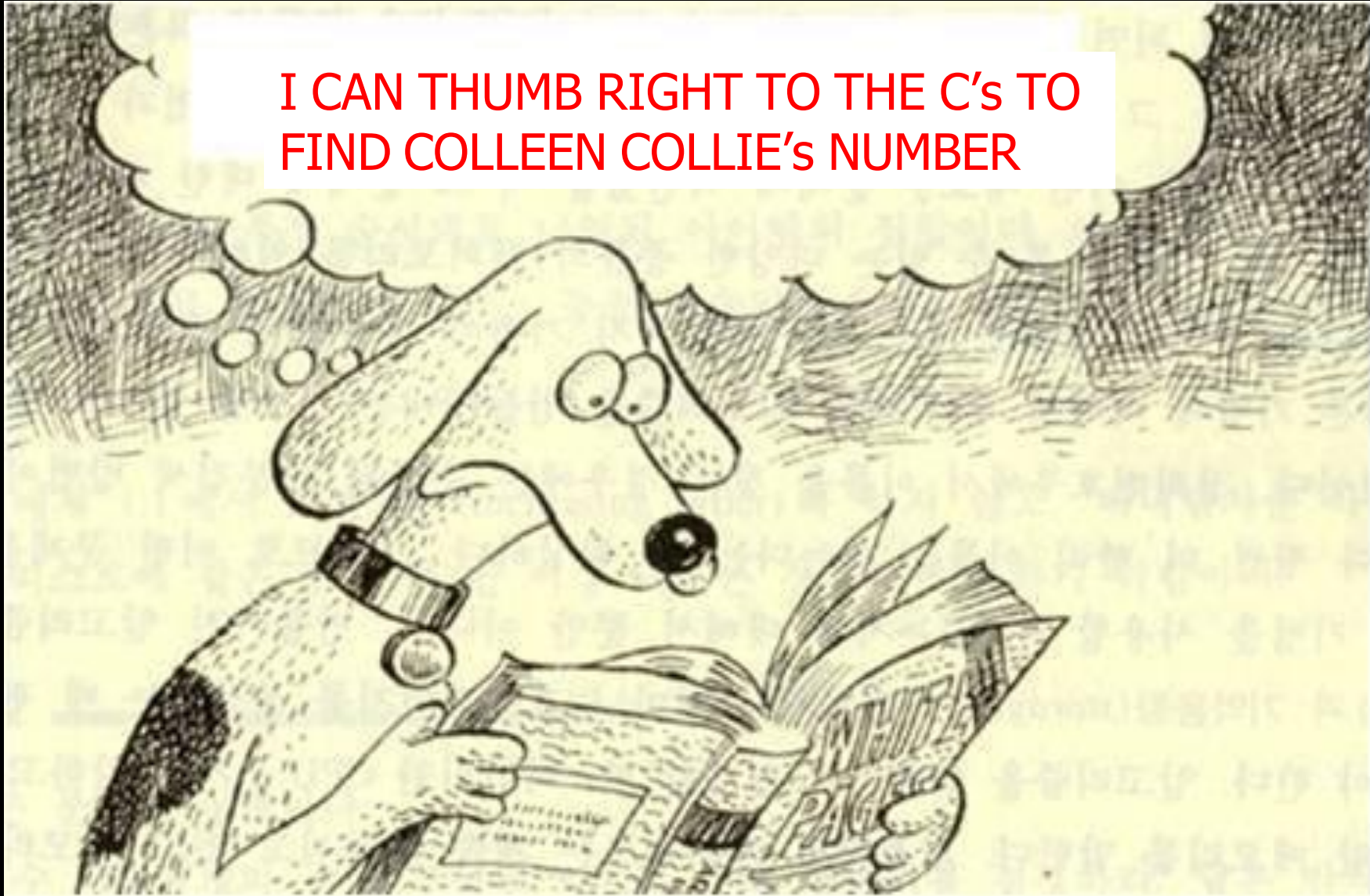
- Analysis

- Efficiency analysis through computational complexity

Acknowledgements

- - Notes adopted from Neapolitan, R. and Naimipour, K., 2015. Foundations of Algorithms.
 - Other Reference acknowledged in slides where appropriate

I CAN THUMB RIGHT TO THE C's TO
FIND COLLEEN COLLIE'S NUMBER



Example

■ Problem

- Determine whether the number x is in the list of S of n numbers. The answer is **yes** if x is in S and **no** if it is not.
- Problem **instance**
 - $S = [10, 7, 11, 5, 13, 8], n = 6, x = 8$
 - Solution to this instance is, “Yes, x is in S ”

■ Algorithm

- Starting with the first item in S , compare x with each item in S in sequence until x is found or S is exhausted. If x is found, answer yes; if x is not found, answer no.

■ Analysis

-  Any **better** algorithm to get the same solution?

Problem Description

- Problem
 - May contain **variables** that **are assigned specific values** in the statement of the problem description.
- Parameters
 - Those **variables** are called **parameters**.
 - For example, to describe a search problem, we need 3 variables: S, n, x .
- Instance
 - If those parameters are specified, we call it an **instance**.
 - $S = [10, 7, 11, 5, 13, 8], n = 6, x = 8$
- Solution to an **instance** of a **problem** is the answer to the question asked by the **problem** in that **instance**.
 - Solution to the above instance is, “**yes, x is in S** ”

Algorithmic Description

- Natural languages
- Programming languages
- Pseudo-code
 - similar, but not identical, to C/C++/Java.
 - Notable exceptions: unlimited array index, variable-sized array, mathematical expressions, use of arbitrary types, convenient control structure, and etc.
- In this course unit, algorithms will be represented by **pseudo-codes** similar to C/C++ & **flow charts**.

Pseudo-code **vs.** C/C++

■ Use of arrays

- In C/C++, starting at 0
- In pseudo-code, arrays indexed by **other integers** are ok.

■ Variable-sized array size

- (e.g) void example (int n)
 - { keytype S[2..n];
 -
 - }
- keytype S[low..high]

Pseudo-code vs. C/C++ ctd..

- Mathematical expression

- `low <= x && x <= high` $\Rightarrow$ `low  $\leq$  x  $\leq$  high`
- `temp = x; x = y; y = temp` $\Rightarrow$ `exchange x and y`

- Use of arbitrary type

- Index
- Number
- Bool

- Control structure

- (e.g.) `repeat (n times) { ... }`

Sequential Search

■ Problem

- Is the key ' x ' in the array S of n keys?

■ Inputs (parameters)

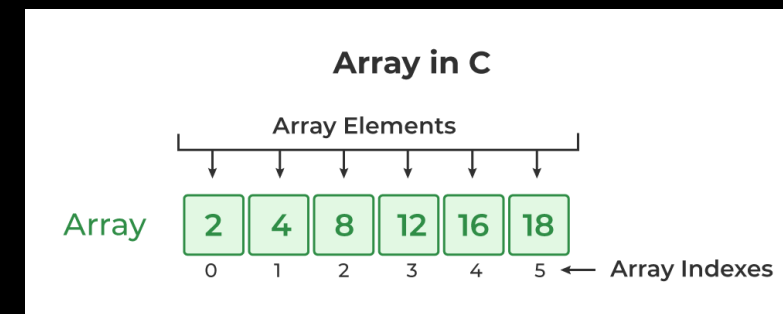
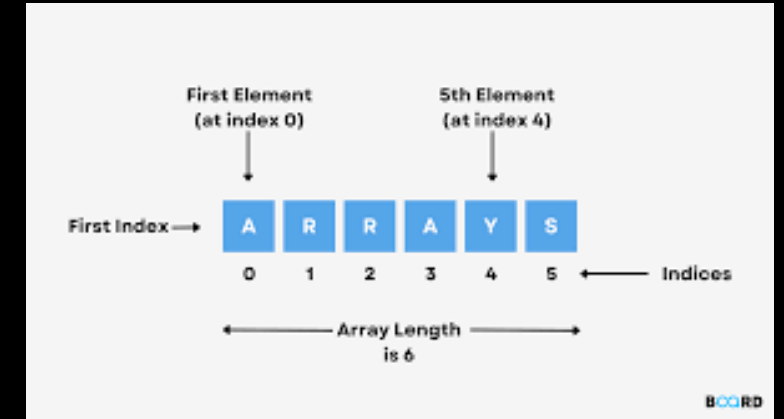
- Positive integer n , array of keys S indexed from 1 to n .

■ Outputs

- The location of x in S . (0 if x is not in S .)

■ Algorithm

- Starting with the first item in S , compare x with each item in S in sequence until x is found or S is exhausted. If x is found, answer yes; if x is not found, answer no.

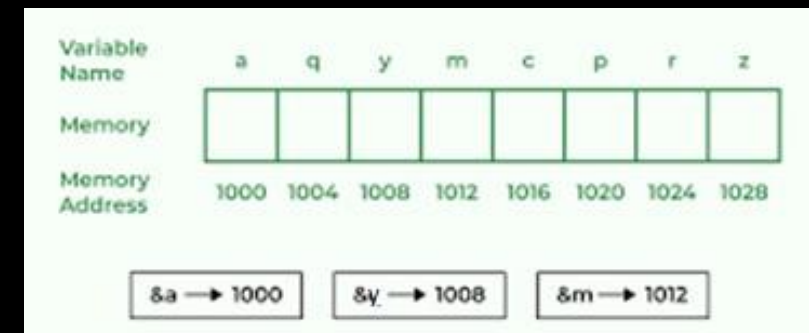


Sequential Search (Pseudo-code)

```
void seqsearch(int n,           // Input (1)
               const keytype S[], // (2)
               keytype x,         // (3)
               Index &location) { // Output

location = 1;
while (location <= n && S[location] != x) {
    location++;
}
if (location > n)
    location = 0;
}
```

'const' keyword in C is used to define constant values that cannot be changed at runtime



Memory Address &

When a variable is created in C, a memory address is assigned to the variable. The memory address is the location of where the variable is stored on the computer.

When we assign a value to the variable, it is stored in this memory address. To access it, use **the reference operator (&)**, and the result represents where the variable is stored:

Review: sequential search

- How many comparisons for searching x in S ?
 - Depends on the location of x in S
 - In worst case, we should compare n times.
 - In best case, 1
- Any *better* algorithm to get the same solution?
 - No, we can't, unless there is an extra information in S .



Binary Search

■ Problem

- Is the key x in the array S of n keys?
- Determine whether x is in the sorted array S of n keys.

■ Inputs (parameters)

- Positive integer n , sorted (non-decreasing order) array of keys S indexed from 1 to n , a key x .

■ Outputs

- The location of x in S . (0 if x is not in S .)

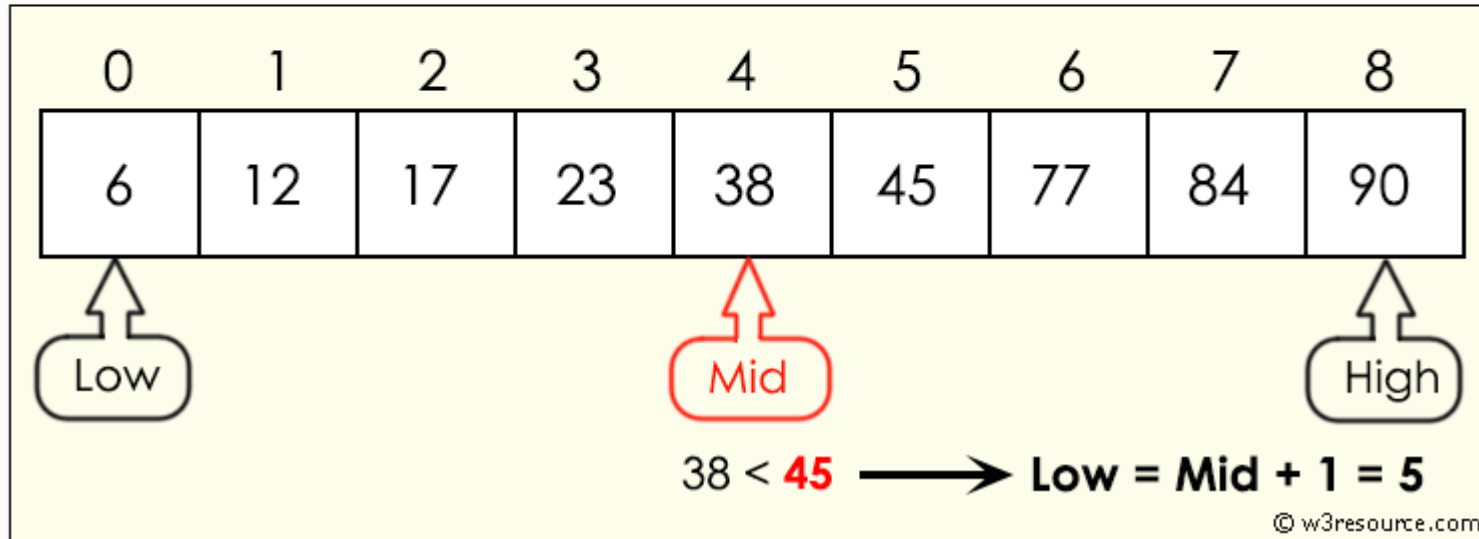
Binary Search

#1

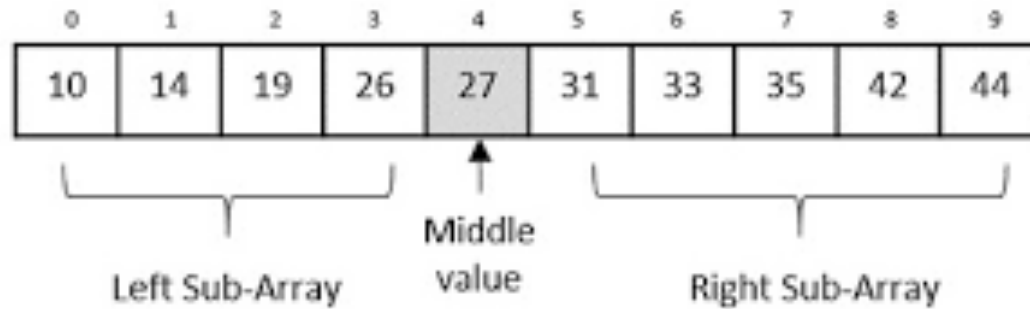
Low	High	Mid
0	8	4

Search (45)

$$mid = \left\lfloor \frac{low + high}{2} \right\rfloor$$



Binary Search



Binary Search Algorithm

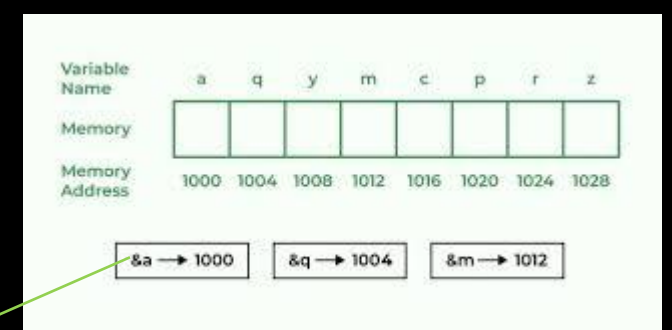


Properties of Binary Search

- Binary search is performed on the sorted data structure for example sorted array.
- Searching is done by dividing the array into two halves
- It utilizes the **divide-and-conquer** (discussed in a later lecture) approach to find an element.



Binary Search (Pseudo-code)



```
void binsearch(int n,           // Input (1)
               const keytype S[], // (2)
               keytype x,       // (3)
               Index &location) { // Output
    index low, high, mid;
    low = 1; high = n;
    location = 0;
    while (low <= high && location == 0) {
        mid = (low + high) / 2; //integer division (keeping
        if (x == S[mid])        //only the whole-number part of
            location = mid;      //the result)
        else if (x < S[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
}
```

Refer: Bin Search Animation

<https://yongdanielliang.github.io/animation/web/BinarySearchNew.html>

Review: binary search algorithm

- How many comparisons for searching x in S ?
 - S is already sorted. We know it.
 - For each statement in a while-loop, the number of searching targets will be half.
 - In worst case, we should compare (?) times.
 - (e.g.) $n = 32$. $S[16]$, $S[16+8]$, $S[24+4]$, $S[28+2]$, $S[30+1]$. $S[32]$
 - In best case, trivial to answer.
- Any *better* algorithm to get the same solution?
 - See Chap. 7-8.

The number of comparisons done by Sequential Search and Binary Search when **x is larger than all the array items**

Array Size	Number of Comparisons by Sequential Search	Number of Comparisons by Binary Search
128	128	8
1,024	1,024	11
1,048,576	1,048,576	21
4,294,967,296	4,294,967,296	33

Fibonacci Sequence

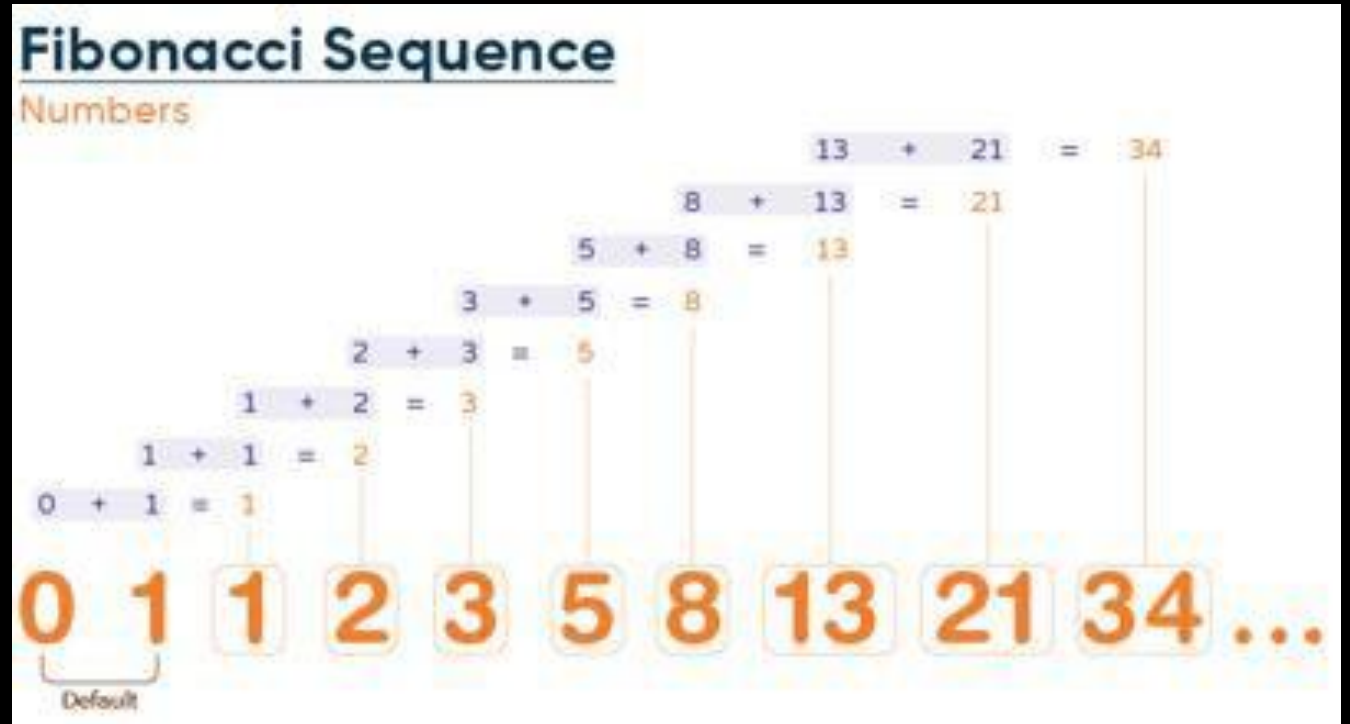
- The Fibonacci Sequence is a number series in which each number is obtained by **adding its two preceding numbers**. It starts with 0 and is followed by 1.
- The numbers in this sequence, known as the Fibonacci numbers, are denoted by F_n .

Fibonacci Sequence Formula

$$F_n = F_{n-1} + F_{n-2}, \text{ for } n > 2$$

here,

- F_n is the n^{th} term number,
- F_{n-1} is the $(n-1)^{\text{th}}$ term number, and
- F_{n-2} is the $(n-2)^{\text{th}}$ term number



Fibonacci Sequence

- The **Fibonacci Sequence** is **the series of numbers**:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

- The next number is found by adding up the two numbers before it:
 - the 2 is found by adding the two numbers before it (1+1),
 - the 3 is found by adding the two numbers before it (1+2),
 - the 5 is (2+3),
 - and so on!
- Example: the next number in the sequence above is $21+34 = 55$

Fibonacci Sequence

First, the terms are numbered from 0 onwards like this:

$n =$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	...
$x_n =$	0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	...

So term number 6 is called x_6 (which equals 8).

Example: the **8th** term is the **7th** term plus the **6th** term:

$$x_8 = x_7 + x_6$$

x_5	x_6	x_7	x_8	x_9	x_{10}
5	8	13	21	34	55

$x_8 = x_7 + x_6$

So we can write the rule:

$$\text{The Rule is } x_n = x_{n-1} + x_{n-2}$$

where:

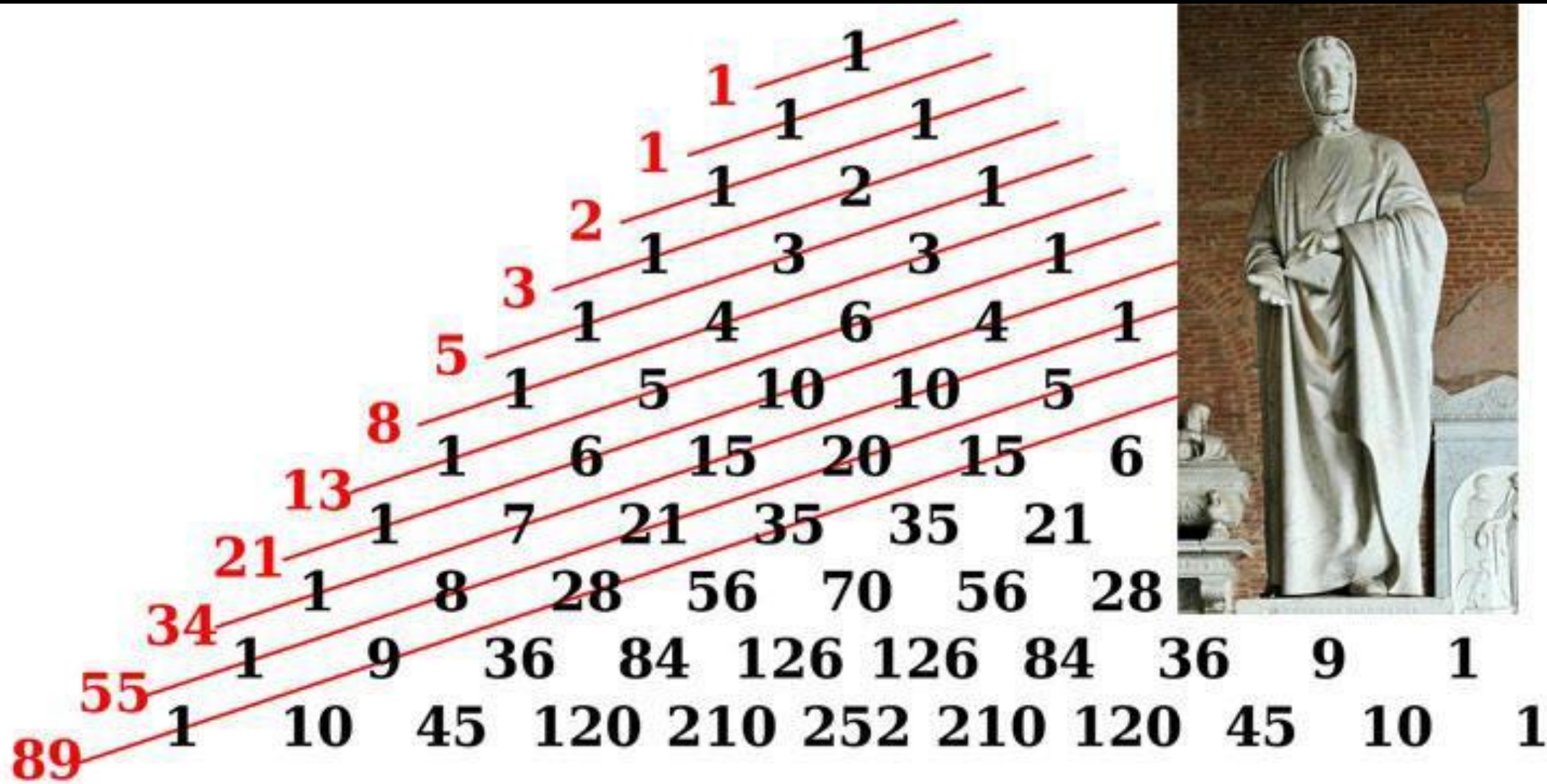
- x_n is term number "n"
- x_{n-1} is the previous term (n-1)
- x_{n-2} is the term before that (n-2)

Example: term 9 is calculated like this:

$$\begin{aligned}x_9 &= x_{9-1} + x_{9-2} \\&= x_8 + x_7 \\&= 21 + 13 \\&= 34^{24}\end{aligned}$$

Fibonacci Sequence

Leonardo Fibonacci. The Italian mathematician, who was born around A.D. 1170, was initially known as Leonardo of Pisa.



https://www.daviddarling.info/encyclopedia/F/Fibonacci_sequence.html

Fibonacci Sequence ctd..

- Here is a longer list:
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811, 514229, ...
- *Can you figure out the **next few numbers**?*

Special Observations on Fib sequence?

Here is the Fibonacci sequence again:

$n =$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	...
$x_n =$	0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	...

There is an interesting pattern:

- Look at the number $x_3 = 2$. Every 3rd number is a multiple of 2 (2, 8, 34, 144, 610, ...)
- Look at the number $x_4 = 3$. Every 4th number is a multiple of 3 (3, 21, 144, ...)
- Look at the number $x_5 = 5$. Every 5th number is a multiple of 5 (5, 55, 610, ...)

And so on (every n th number is a multiple of x_n).

An odd fact:

The sequence goes even, **odd, odd**, even, **odd, odd**, even, **odd, odd**, ... :

0	1	1	2	3	5	8	13	21	34	55	89	144	233	377	610	...
---	---	---	---	---	---	---	----	----	----	----	----	-----	-----	-----	-----	-----

Why?

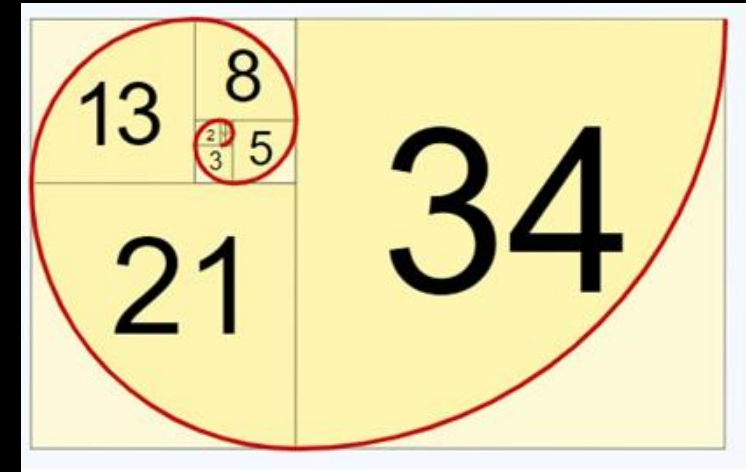
Because adding **two odd numbers produces an even number**, but adding **even and odd (in any order) produces an odd number**.

Fibonacci Sequence ctd..

Phi (uppercase Φ /lowercase ϕ), is the 21st letter of the Greek alphabet, used to represent the "ph" sound in Ancient Greek.: **Phi is an irrational mathematical constant, approximately 1.618..**, and is often denoted by the Greek letter ϕ .

- Other commonly used names for Phi are: **Golden Mean, Extreme and Mean Ratio, Divine Proportion and Golden Ratio.**
- Question : (i) **Does Phi ϕ a naturally occurring ratio which exhibits aesthetically pleasing properties?**
What is Golden Ratio?

When we **make squares with those widths**, we get a nice spiral:



Workout activities in Golden Ratio

Refer <https://www.mathsisfun.com/numbers/golden-ratio.html>

Golden Ratio - ϕ

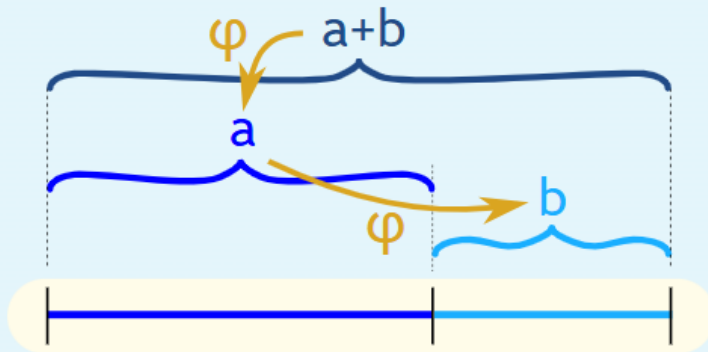
The Idea Behind It

We find the golden ratio when we divide a line into two parts so that:

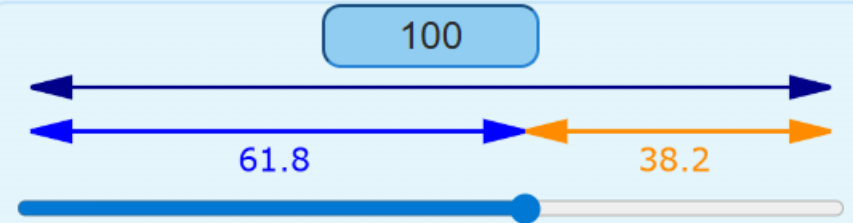
the whole length divided by the long part

is also equal to

the long part divided by the short part



$$\frac{a+b}{a} = \frac{a}{b} = 1.618... = \phi$$



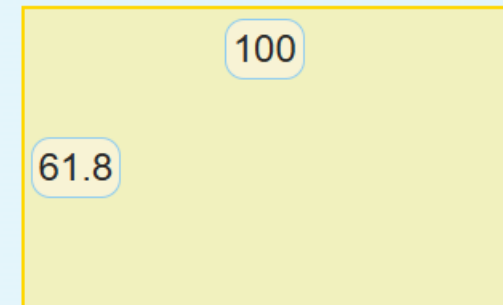
$$\frac{100}{61.8} = 1.618$$

$$\frac{61.8}{38.2} = 1.618$$

They are the same!
The Golden Ratio!

=

Landscape

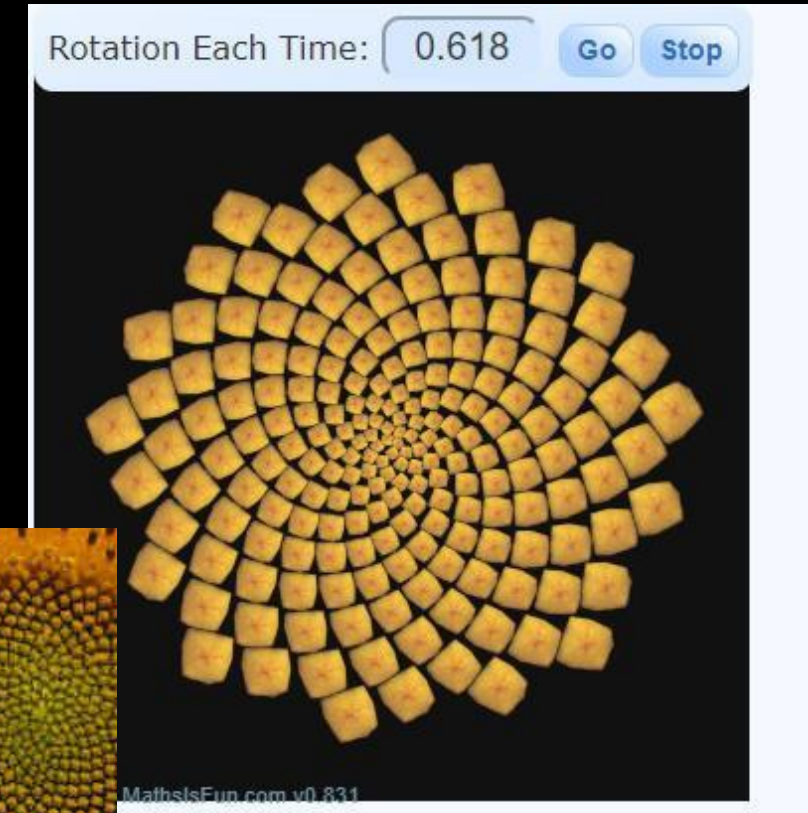
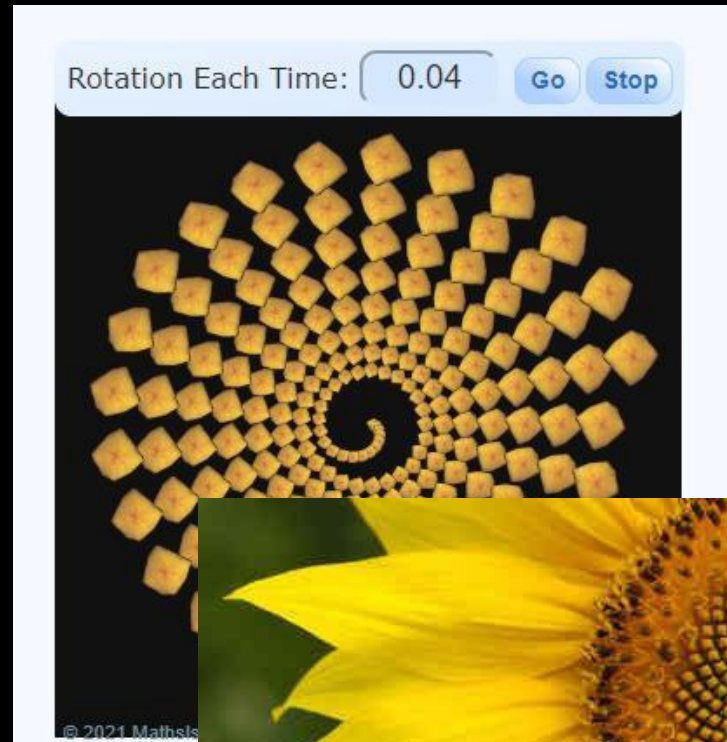


[PI visit and try interactive activity-> Golden Ratio](https://www.mathsisfun.com/numbers/golden-ratio.html)

<https://www.mathsisfun.com/numbers/golden-ratio.html>

Refer : Nature, The Golden Ratio, and Fibonacci too ...

<https://www.mathsisfun.com/numbers/nature-golden-ratio-fibonacci.html>



<https://www.mathsisfun.com/numbers/fibonacci-sequence.html>

Natural Phenomena and Mathematics

- This interesting behavior is not just found in sunflower seeds.
- **Leaves, branches and petals can grow in spirals, too.**
- Why? So that new leaves don't block the sun from older leaves, or so that the maximum amount of rain or dew gets directed down to the roots.
- **Question:** What are Golden Ratio and Golden Angle? What is the relationship between Golden Ratio and Fibonacci Sequence?



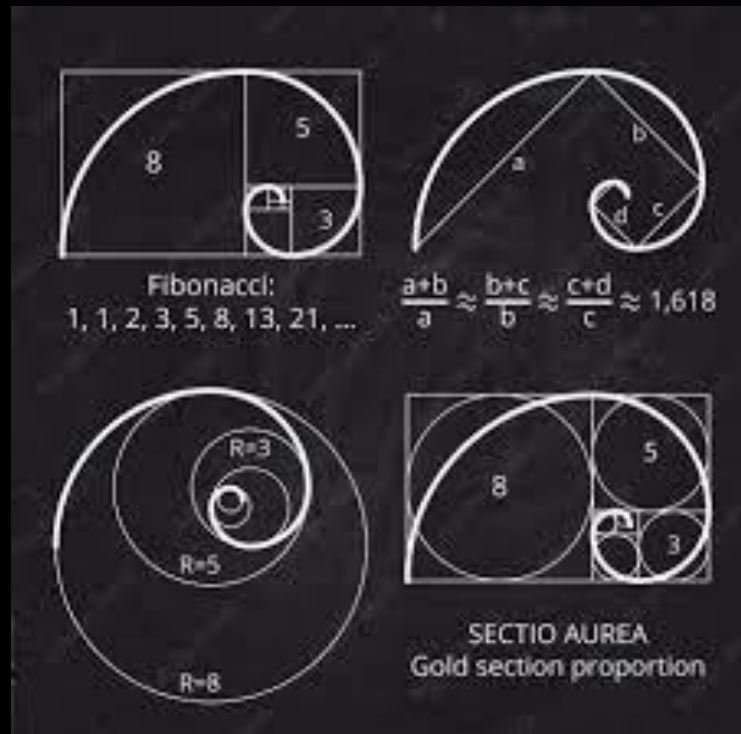
<https://www.mathsisfun.com/numbers/fibonacci-sequence.html>



Fibonacci Numbers and Golden Ratio

There is a special relationship between the Golden Ratio and Fibonacci Numbers (0, 1, 1, 2, 3, 5, 8, 13, 21, ... and so on, *each number is the sum of the two numbers before it*).

When we take **any two successive** (*one after the other*) Fibonacci Numbers, their ratio is very **close to the Golden Ratio**:

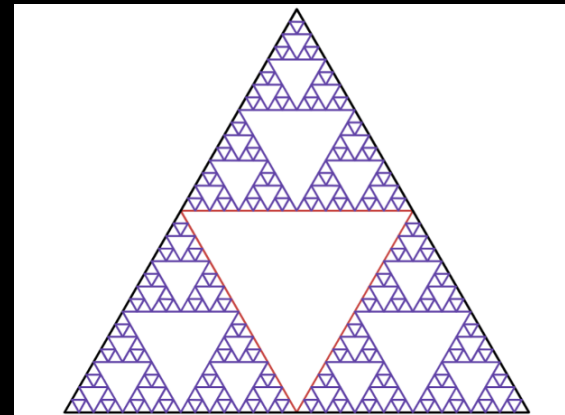
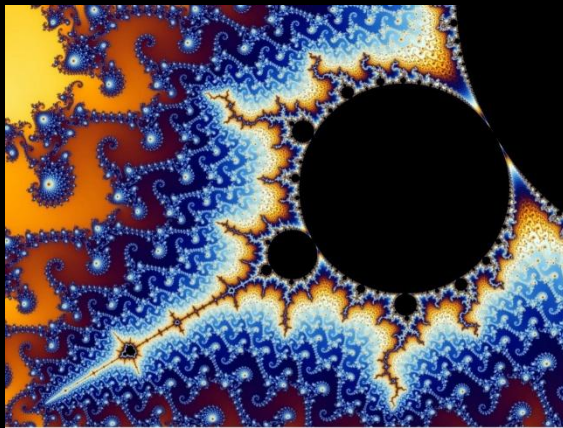


A	B	B / A
2	3	1.5
3	5	1.666666666...
5	8	1.6
8	13	1.625
13	21	1.615384615...
...	...	...
144	233	1.618055556...
233	377	1.618025751...

Extra: Additional Observations - Nature and mathematics



Beginning Game Development: Terrain with Perlin Noise



The Art of Nature: Fractals, Fibonacci, and the Golden Ratio – Castor Magazine

The Fibonacci Sequence

- Problem

- Determine the n -th term in the Fibonacci sequence.

$$f_0 = 0$$

$$f_1 = 1$$

$$f_n = f_{n-1} + f_{n-2} \quad \text{for } n \geq 2$$

- Inputs

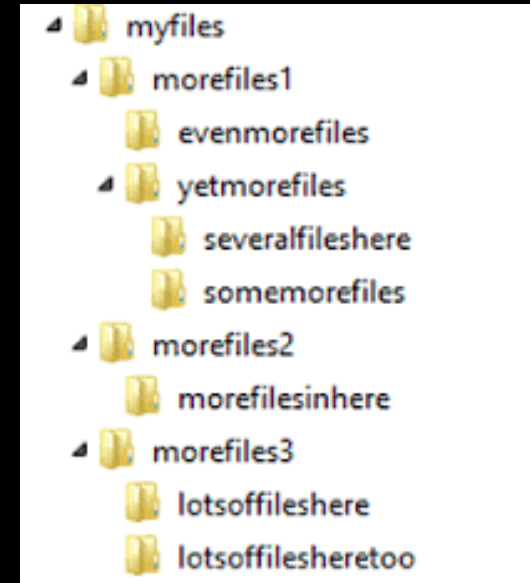
- A non-negative integer n .

- Outputs

- The n -th term of the Fibonacci sequence.

Recursive?

- **Recurrence Relation in Mathematics?**
- In English "recur" means to occur again (and again)
- Where a number is calculated from numbers before it in a sequence.
- Example: the factorial function has each number calculated from the previous number in this way:
 - $n! = n(n-1)!$
 - So $10! = 10 \times 9!$, and $9! = 9 \times 8!$, down to $1! = 1$
- The Fibonacci Sequence is another example.
- Another example from **Data**? **recursive data** found in every computer: its **filesystem in OS**.



Recursion

Recursion is a programming and mathematical technique where a function solves a problem by calling itself on **smaller versions of the same problem** until it reaches a **stopping condition** called the **base case**.

Simple Example: Factorial

The factorial of a number n is: $n! = n \times (n-1)!$,
 $0! = 1$

For factorial(5):

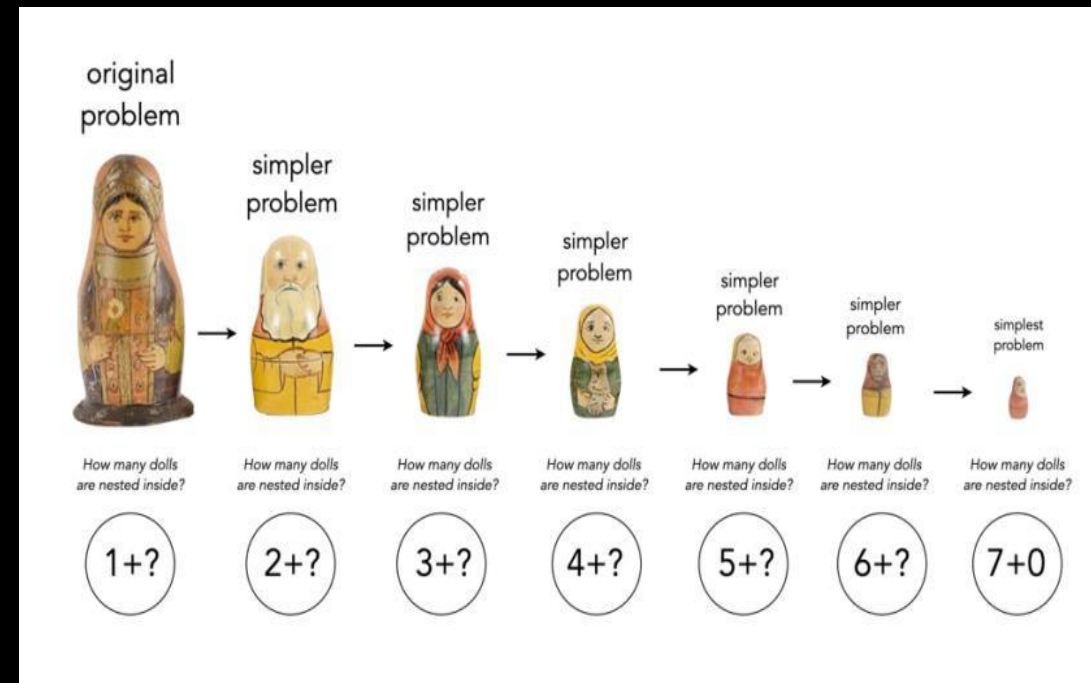
5 * factorial(4)
5 * 4 * factorial(3)
5 * 4 * 3 * factorial(2)
5 * 4 * 3 * 2 * factorial(1)
5 * 4 * 3 * 2 * 1

Key Parts of Recursion

- Base Case: stops recursion
- Recursive Case : function calls itself with a smaller/simpler input

Thinking Recursively?

- Another good example of thinking recursively can be found in the **Russian nesting dolls**, or Matryoshka dolls.
- How do determine how many dolls are inside the doll?
- The best approach is to **continuously open the outer doll, add it to the count**, and then solve the problem of how many dolls remain nested?
- At **each opening, the problem becomes smaller**, because there are fewer dolls.
- At **some point the problem must be solved, because no more dolls can be nested** (ignore the differences in scale of the dolls).



Recursive Problem Solving

- Is this recursive problem solving? In some respects yes, it is.
- Each time the outer doll is separated, the **problem now becomes smaller**, both figuratively and literally.
- When we reach the doll which can not be separated, the problem is solved.
- Here  is how it could be described algorithmically (in a round-about sort of way):

`doll-count is 0`

`count-NestingDolls`(doll)

`begin`

`if (doll can be separated)`

`smaller-doll = separate(doll)`

`increase doll-count by 1`

`perform count-NestingDolls(smaller-doll)`

`otherwise`

`increase doll-count by 1`

`done`

`end`

Ex. Write the Loop Version of the above Algorithm(i.e no recursion)

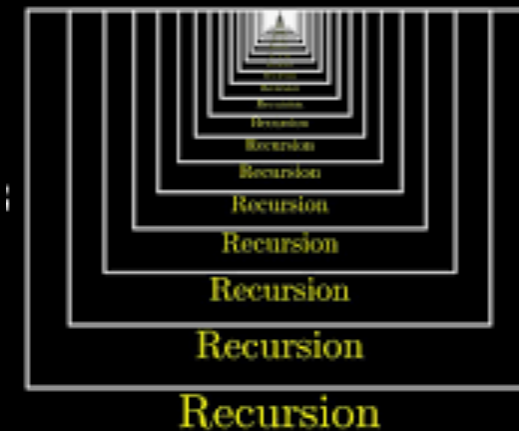
Recursive?

- Applying a rule or formula to its own result, again and again.
- Example: 1, 2, 4, 8, 16, 32, ...
-
- **The base case**
 - You might ask, ``Can a recursive algorithm go on **forever**?''
 - Every recursive algorithm *must* have a case in which it does not recurse -- called the **base case**.

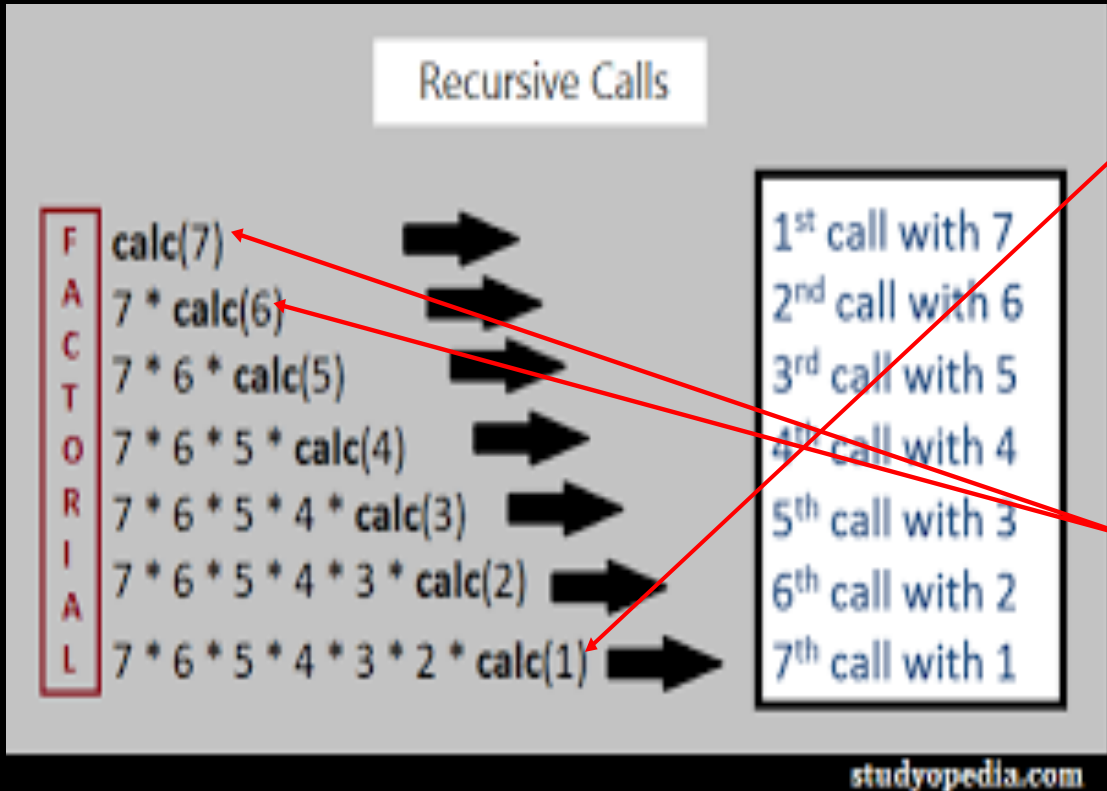
Common Uses

- Tree and graph traversal
- Divide-and-conquer algorithms
- Backtracking
- Dynamic programming
- Mathematical sequences (Fibonacci, factorial)

<https://www.mathsisfun.com/definitions/recursive.html>



Structure of Recursive Implementations



- A recursive implementation always has two parts:
- **base case**, which is the **simplest, smallest instance of the problem**, that can't be decomposed any further. Base cases often correspond to emptiness – the empty string, the empty list, the empty set, the empty tree, zero, etc.
- **recursive step**, which **decomposes a larger instance of the problem into one or more simpler or smaller instances that can be solved by recursive calls**, and then **recombines the results** of those subproblems to produce the solution to the original problem.

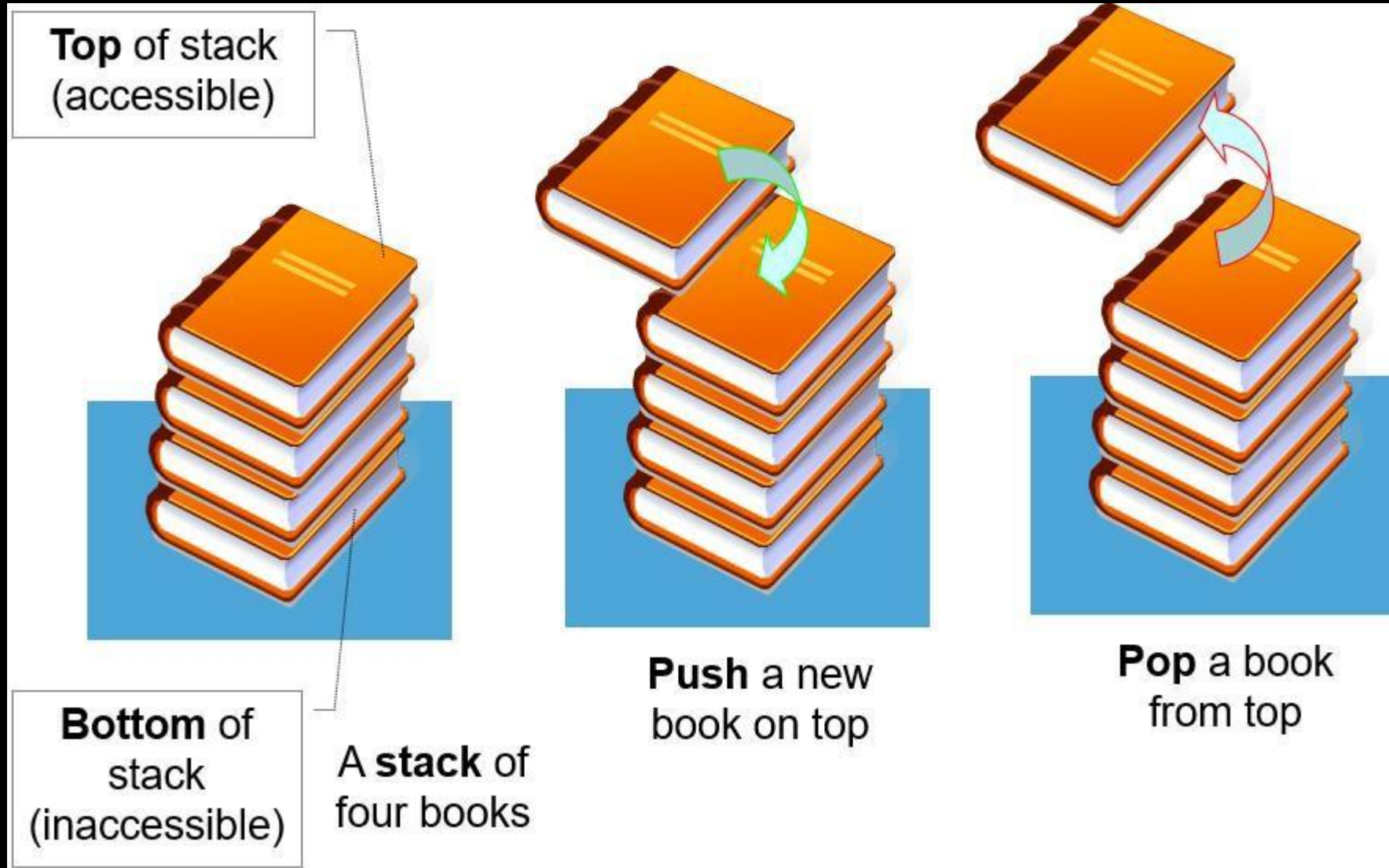
Refer: Reading 10: Recursion

<https://web.mit.edu/6.005/www/fa15/classes/10-recursion/#:~:text=implementation%20can%20solve.-,Common%20Mistakes%20in%20Recursive%20Implementations,the%20base%20cases%20are%20covered.>

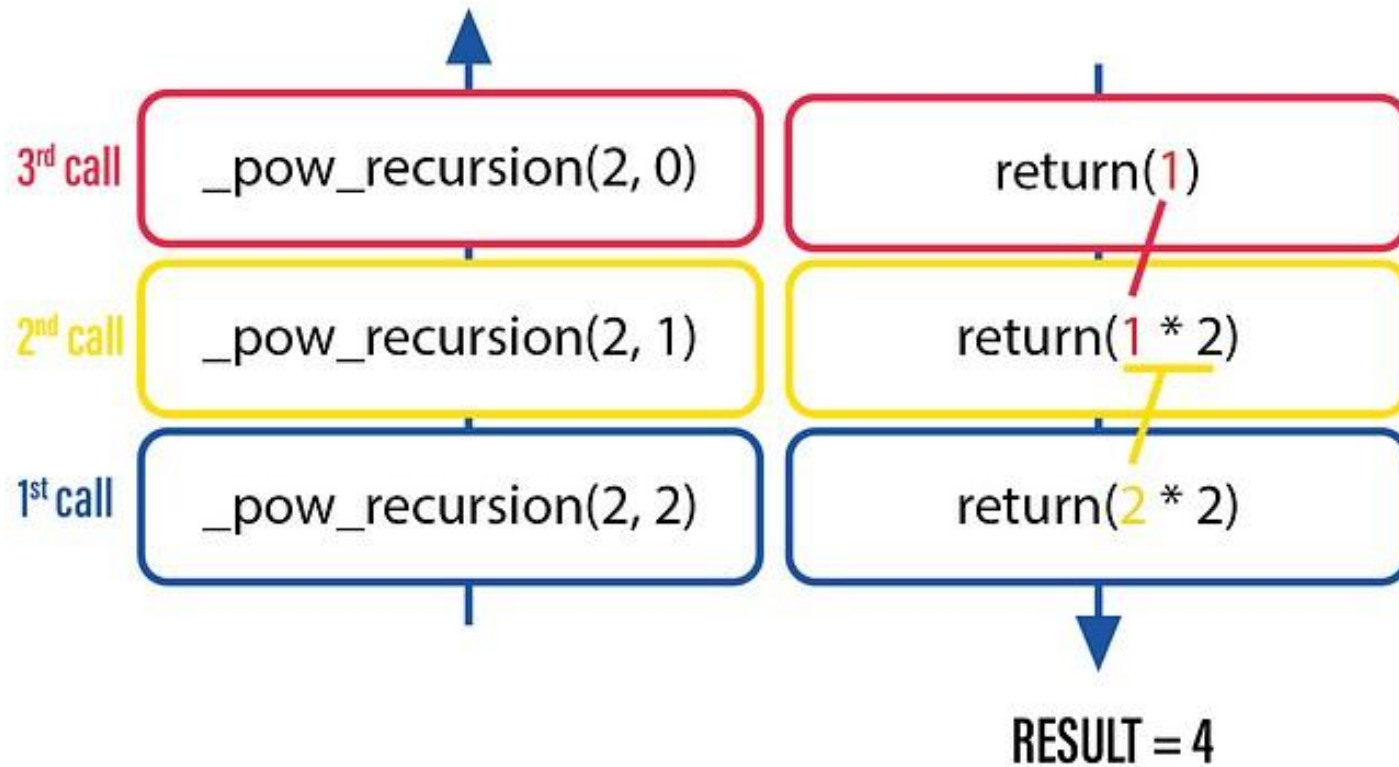
Properties of Recursion:

- Recursion has some important properties. **Some** of which are mentioned below:
 1. The primary property of recursion is the **ability to solve a problem by breaking it down into smaller sub-problems, each of which can be solved in the same way.**
 2. A recursive function **must have a base case** or stopping criteria to **avoid infinite recursion.**
 3. Recursion involves calling the same function within itself, **which leads to a call stack.**
 4. Recursive functions **may be less efficient than iterative solutions in terms of memory and performance.**

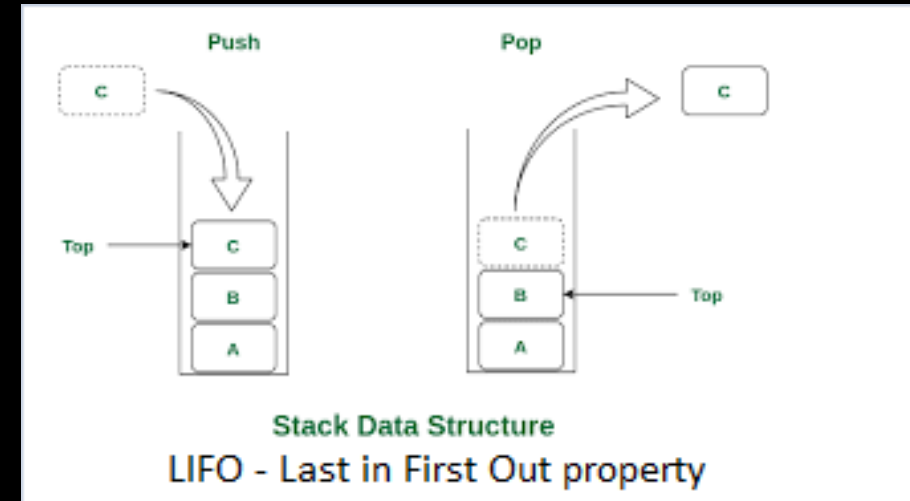
Stack?



Recursion managed with a 'Stack'?

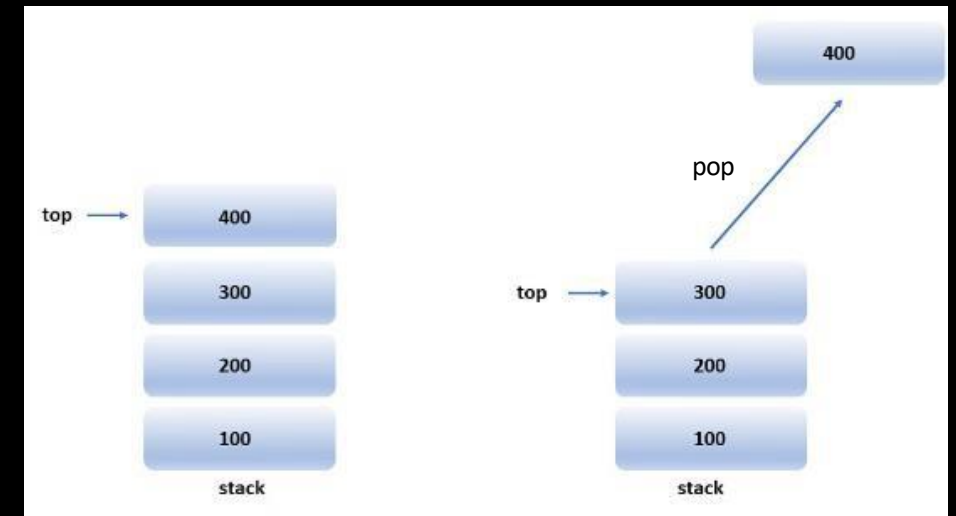
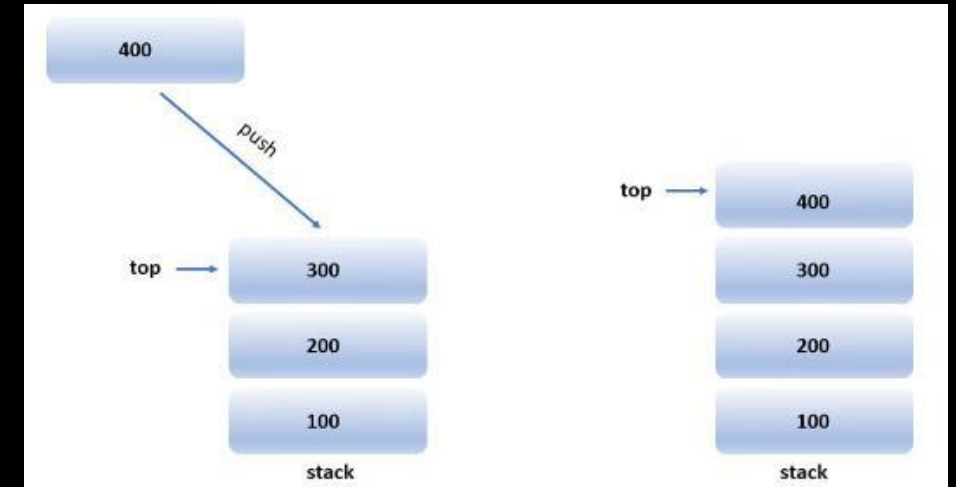
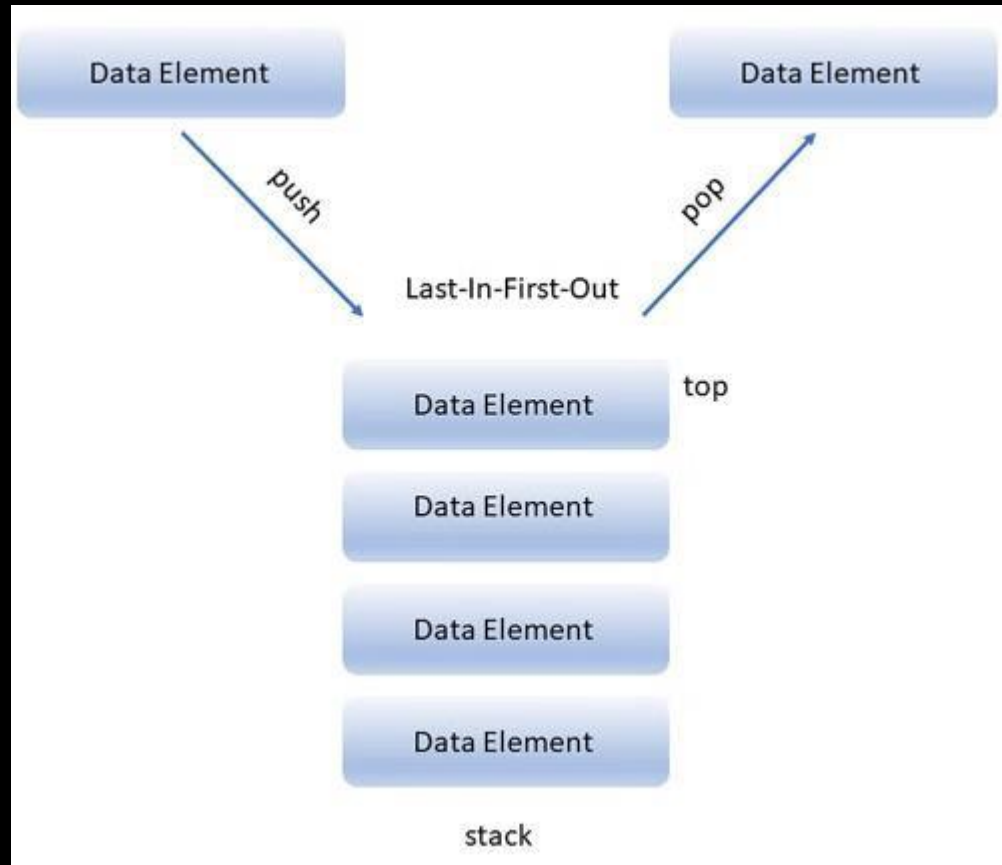
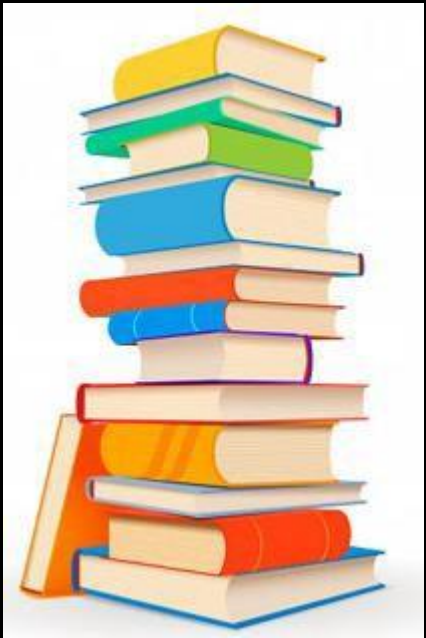


What is a Stack Data Structure?



https://medium.com/@4318_26766/recursion-and-how-it-works-on-the-stack-bdcde726331

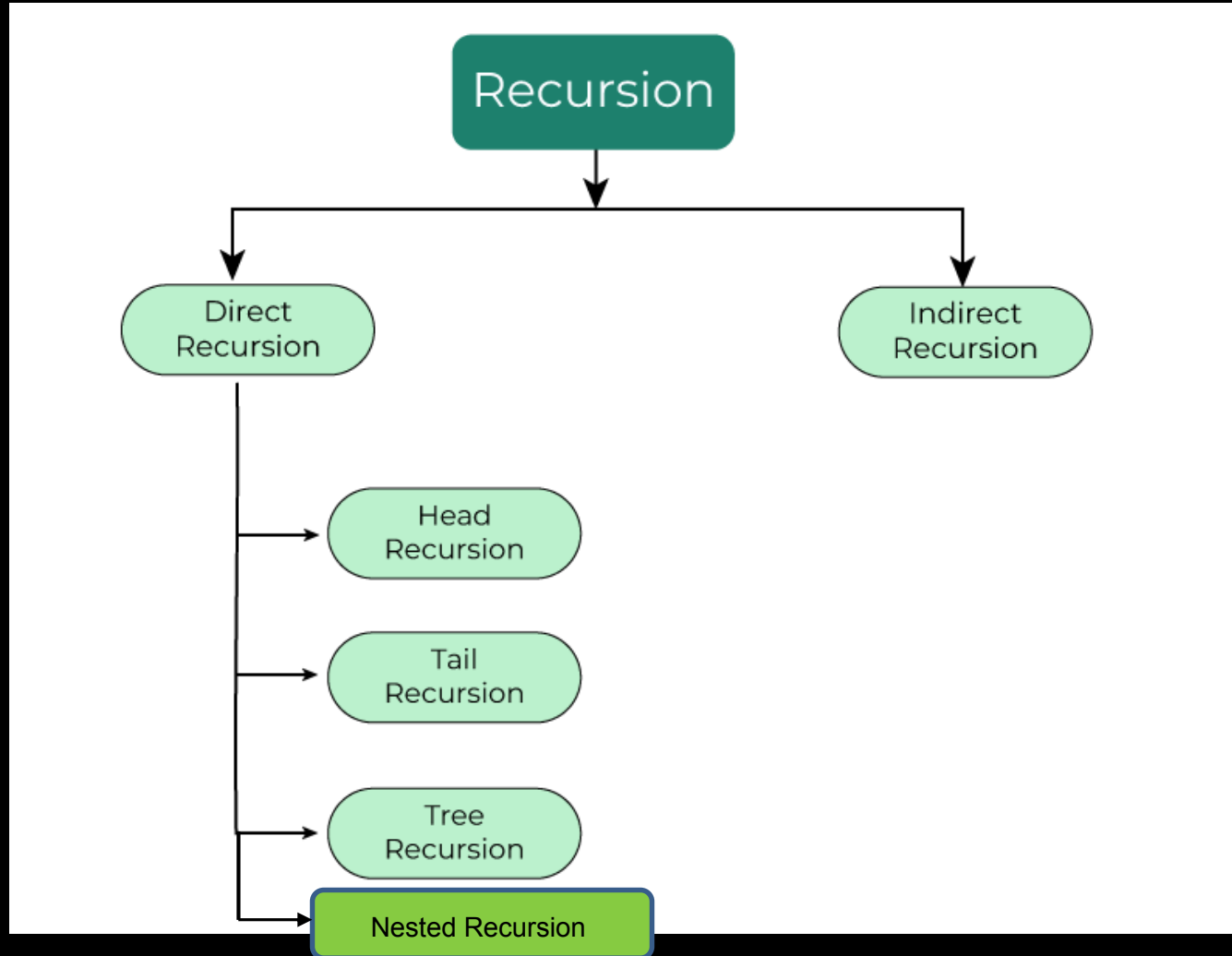
Stack with Last in First Out(LIFO) property?



Refer Stack in Data Structure: Master LIFO Concepts Easily

<https://www.simplilearn.com/tutorials/data-structure-tutorial/stacks-in-data-structures>

Types of Recursion



Types of Recursion:

1. **Direct recursion:** When a function is called within itself **directly** it is called direct recursion. This can be **further categorized into four** types:
 - i. Tail recursion,
 - ii. Head recursion,
 - iii. Tree recursion and
 - iv. Nested recursion.
2. **Indirect recursion:** Indirect recursion occurs when a function calls another function that eventually calls the original function and it forms a cycle.

Types of Recursion

Head recursion → recursive call happens first, work happens later.

Tail recursion → work happens first, recursive call happens last.

Tree recursion → one function call creates multiple recursive calls.

Nested recursion → a recursive call is used inside another recursive call.

Types of Recursion: Direct recursion

- Direct recursion is a type of recursion in which **a function directly calls itself during its execution**.
- The function solves a smaller subproblem and then calls itself with the **reduced subproblem until it reaches a base case that terminates** the recursion.
- Direct recursion involves a straightforward and explicit self-reference within the function's body.

TRACING RECURSIVE ALGORITHMS:

How to use a trace table and a tree to trace recursive functions

<https://www.youtube.com/watch?v=pZOjRmjXskQ>

```
factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

the `factorial()` function calls itself with a smaller value `n - 1` until it reaches the base case where `n` is equal to 0.

This recursive approach calculates the factorial of a given number by multiplying it with the factorial of the preceding number.

```
def factorial(x):
    if x == (0) #Base Case
        return 1
    else:
        return x * (factorial(x - 1))

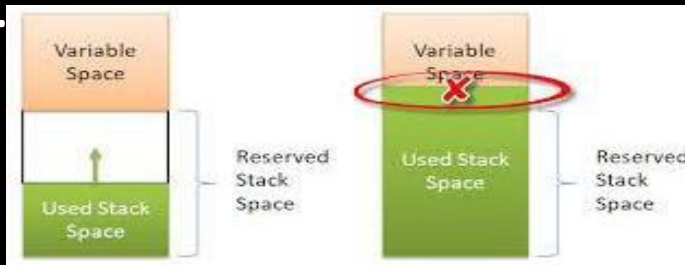
ans = factorial(5)
print (ans)
```

Level	X	IF X = 0	Return
0	5	F	
1	4	F	
2	3	F	
3	2	F	
4	1	F	
5	0	T	1
4	1	F	1
3	2	F	2
2	3	F	6
1	4	F	24
0	5	F	120
OUTPUT			120

Direct Recursion - Pros and Cons:

offers some **advantages** in problem-solving and algorithm design:

- **Simplicity**: Direct recursion often provides a **straightforward and intuitive solution** for problems that exhibit self-similar subproblems.
- **Readability**: Recursive functions can **often express the problem-solving logic more clearly and concisely**, making the code easier to understand.



However, also has some **drawbacks**:

- **Memory Usage**: Recursive function calls consume memory as each call creates a new stack frame. If the recursion depth is large, it may lead to **stack overflow** errors.
- **Performance Overhead**: Recursive calls involve function call overhead, which can impact performance compared to iterative approaches.
- **Tail Recursion** (covered in this note later)
Optimization: Direct recursion may not benefit from tail recursion optimization, where **the recursive call is the last operation in the function**. This optimization eliminates the need for maintaining multiple stack frames, enhancing performance.

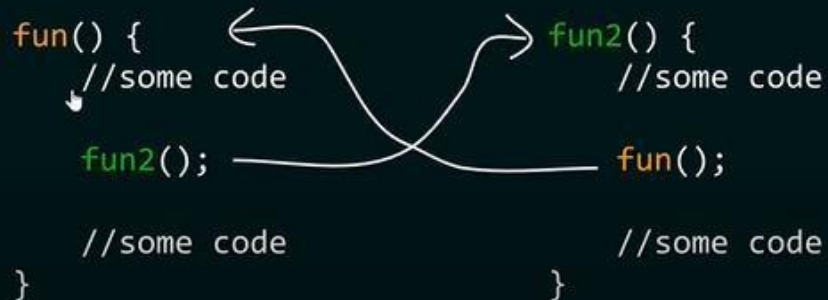
Types of Recursion: Indirect Recursion

Example Implementation:

- Indirect recursion is a type of recursion in which a function calls another function(s).
- The **chain** of function **calls leads back to the original function, creating a cycle.**
- In indirect recursion, there is a circular dependency among multiple functions, where each function calls another function(s) in a sequence **until the base case is reached.**

```
void function1(int n);  
void function2(int n) {  
    if (n > 0) {  
        cout << n << " ";  
        function1(n - 1);  
    }  
}  
  
void function1(int n) {  
    if (n > 1) {  
        cout << n << " ";  
        function2(n / 2);  
    }  
}  
  
int main() {  
    function1(20);  
    return 0;  
}
```

Structure of Indirect recursion:



Indirect Recursion - Pros and Cons:

offers certain **advantages** :

- **Problem Decomposition**: Indirect recursion can be useful for breaking down a complex problem into **smaller, interdependent subproblems**. Each function focuses on solving a specific part of the problem.
- **Code Modularity**: By dividing the problem-solving logic across multiple functions, the code can be organized and modularized, improving readability and maintainability.

<https://intellipaath.com/blog/recursion-in-data-structure/>

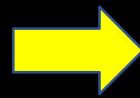
also has some **drawbacks**:

- **Complexity**: Indirect recursion can introduce additional complexity **due to the interdependencies between functions**. This complexity can make code harder to understand and debug.
- **Execution Order**: The execution order of functions in indirect recursion is crucial. **Incorrect sequencing or missing base cases can lead to infinite loops or incorrect results.**
- **Performance Overhead**: Similar to direct recursion, indirect recursion can **incur function call overhead and memory consumption**. Care must be taken to avoid excessive recursive calls.

Tail Recursion

- Tail recursion is defined as a recursive function in which the **recursive call is the last statement that is executed by the function**.
- Hence nothing is left to execute after the recursion call.
 - e.g Consider a simple **function that adds the first N natural** numbers. (e.g. $\text{sum}(5) = 0 + 1 + 2 + 3 + 4 + 5 = 15$).

```
function recsum(x) {  
    if (x === 0) {  
        return 0;  
    } else {  
        return x + recsum(x - 1);  
    }  
}
```



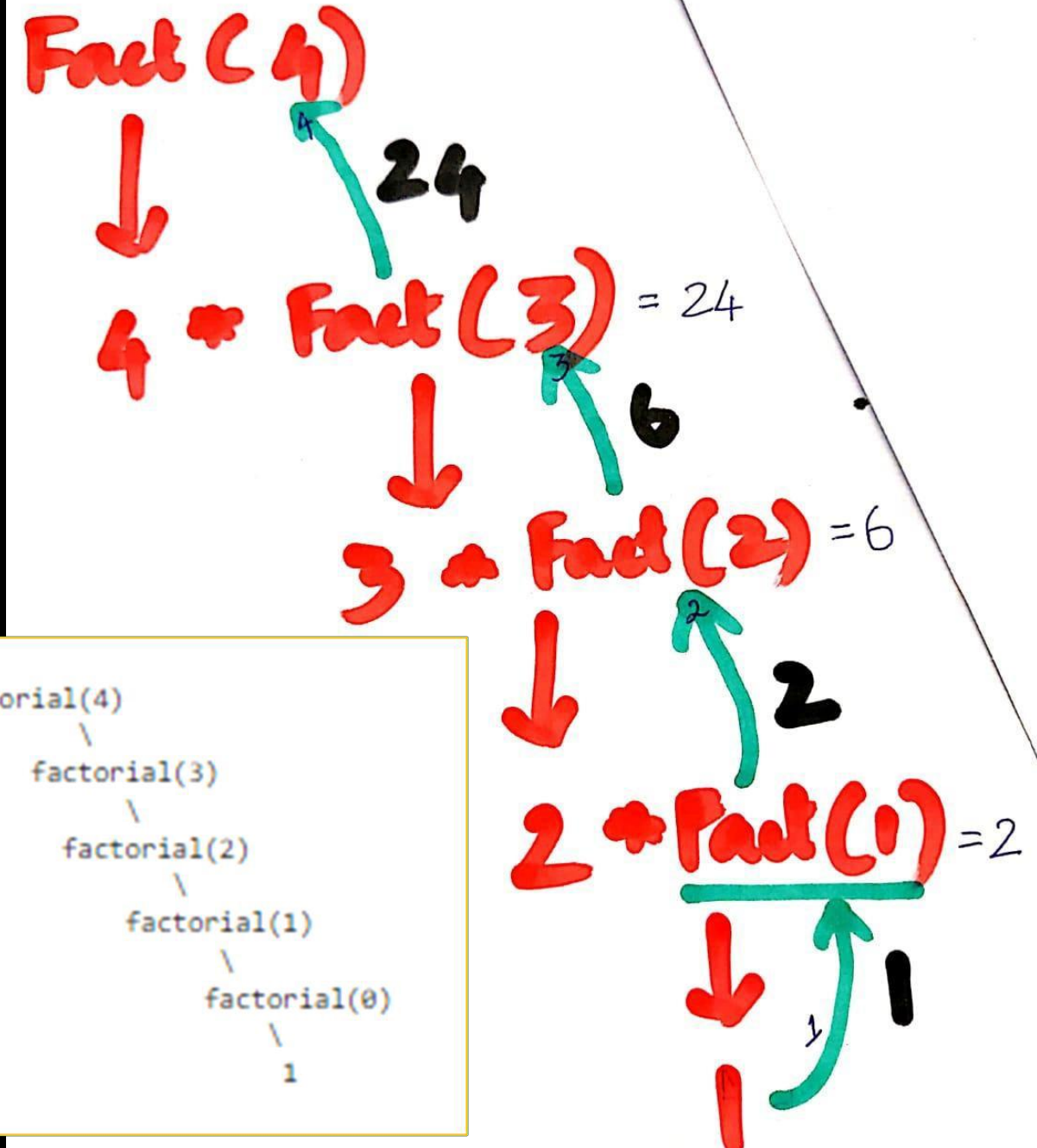
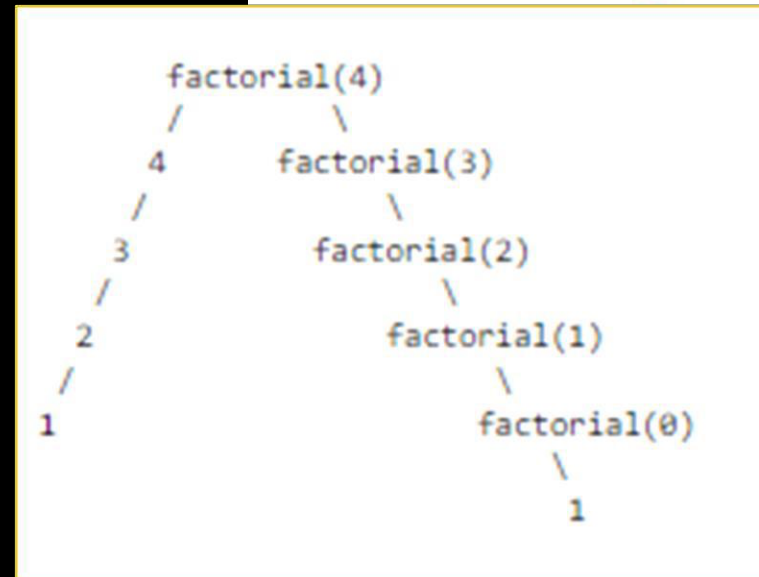
```
recsum(5)  
5 + recsum(4)  
5 + (4 + recsum(3))  
5 + (4 + (3 + recsum(2)))  
5 + (4 + (3 + (2 + recsum(1))))  
5 + (4 + (3 + (2 + (1 + recsum(0)))))  
5 + (4 + (3 + (2 + (1 + 0))))  
5 + (4 + (3 + (2 + 1)))  
5 + (4 + (3 + 3))  
5 + (4 + 6)  
5 + 10  
15
```

e.g. Tail Recursion – Factorial

-
- $\text{fact}(4) = 4 * 3 * 2 * 1 = 24$

```
public static int fact(int n){  
    if(n <=1)  
        return 1;  
    else  
        return n * fact(n-1);  
}
```

<https://stackoverflow.com/questions/33923/what-is-tail-recursion>



Head Recursion

- Head Recursion: If a recursive function calling itself and that **recursive call is the first statement in the function** then it's known as Head Recursion. **There is no statement, no operation before the call.**
- The function does not have to process or perform any operation at the time of calling and **all operations are done at returning time.**

```
// Recursive function
void fun(int n)
{
    if (n > 0) {

        // First statement in the function
        fun(n - 1);

        printf("%d ", n);
    }
}

// Driver code
int main()
{
    int x = 3;
    fun(x);
    return 0;
}
```

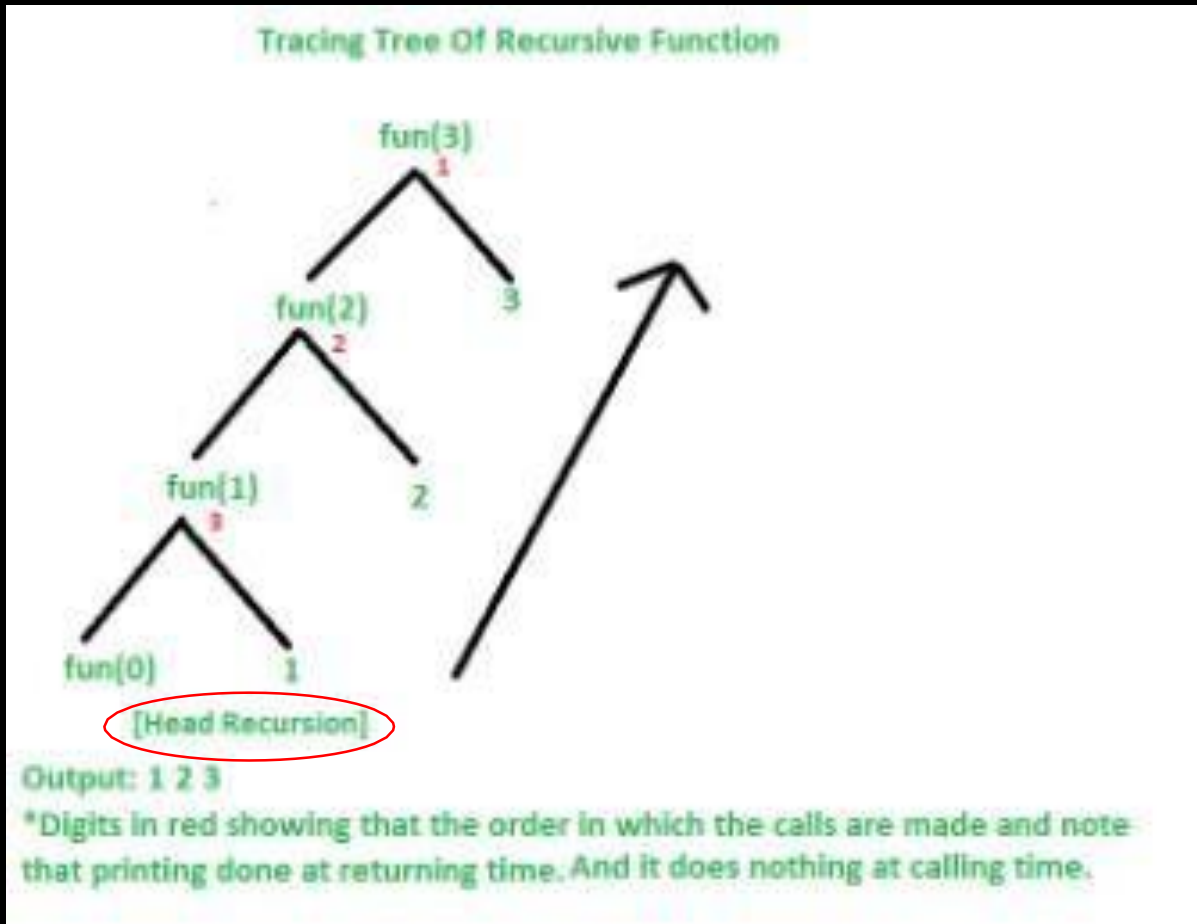
Output

1 2 3

Refer: <https://www.youtube.com/watch?v=wRdM1Z-A35Y>

Head Recursion

- Let us understand the example by **tracing tree of recursive function**.



Output

1 2 3

Head Recursion : Loop version

- Loop version of Head Recursion

```
// Converting Head Recursion into Loop

#include <stdio.h>

// Recursive function
void fun(int n)
{
    int i = 1;
    while (i <= n) {
        printf("%d ", i);
        i++;
    }
}

// Driver code
int main()
{
    int x = 3;
    fun(x);
    return 0;
}
```

Output

1 2 3

Tree Recursion

- To understand Tree Recursion let's first understand Linear Recursion.
- If a recursive function calling itself for one time then it is known as Linear Recursion.
- Otherwise if a recursive function calling itself for more than one time then it's known as Tree Recursion.

```
fun(n)
{
    // some code
    if(n>0)
    {
        fun(n-1); // Calling itself only once
    }
    // some code
}
```

```
// C program to show Tree Recursion

#include <stdio.h>

// Recursive function
void fun(int n)
{
    if (n > 0) {
        printf("%d ", n);

        // Calling once
        fun(n - 1);

        // Calling twice
        fun(n - 1);
    }
}

// Driver code
int main()
{
    fun(3);
    return 0;
}
```

Tree Recursion

- Ex Workout Tracing Tree of Tree Recursion
- Refer
 - <https://dotnettutorials.net/lesson/tree-recursion-in-c/>

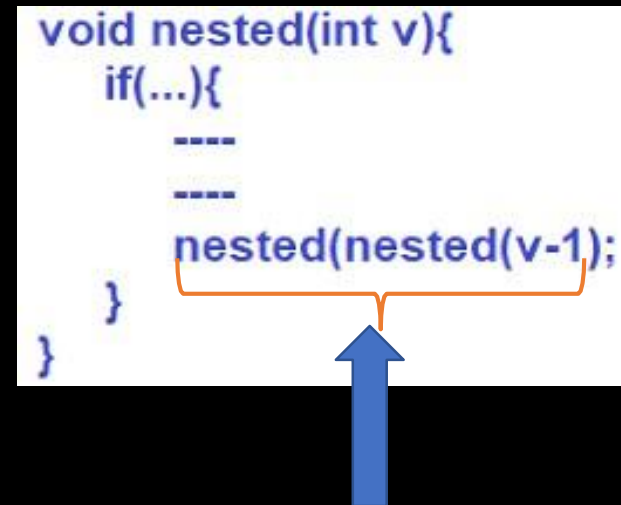
Nested Recursion

- In this recursion, a **recursive function** will pass the **parameter** as a recursive call.
- That means “**recursion inside recursion**”. Let see the example to understand this recursion.

```
#include <stdio.h>
int fun(int n)
{
    if (n > 100)
        return n - 10;

    // A recursive function passing parameter
    // as a recursive call or recursion
    // inside the recursion
    return fun(fun(n + 11));
}

// Driver code
int main()
{
    int r;
    r = fun(95);
    printf("%d\n", r);
    return 0;
}
```



The diagram shows a code snippet for a function named `nested`. The function signature is `void nested(int v){`. Inside, there is an `if(...){` block. Below the `if` block, there are two dashed lines. Then, the line `nested(nested(v-1));` is shown. An orange bracket is drawn under the `nested(v-1)` part of this line. A large blue arrow points upwards from the bottom center towards the bracketed part of the code.

```
void nested(int v){
    if(...){
        ----
        ----
        nested(nested(v-1));
    }
}
```

Applications of Recursion

- Recursion is used in many fields of computer science and mathematics, which includes:
 1. **Searching and sorting algorithms**: Recursive algorithms are used to search and sort data structures like **trees and graphs**.
 2. **Mathematical calculations**: Recursive algorithms are used to solve problems such as **factorial, Fibonacci sequence**, etc.
 3. **Compiler design**: Recursion is used in the design of compilers to parse and analyze programming languages.
 4. **Graphics**: many computer graphics algorithms, such as fractals and the Mandelbrot set, use recursion to generate complex patterns.
 5. **Artificial intelligence**: recursive neural networks are used in natural language processing, computer vision, and other AI applications.

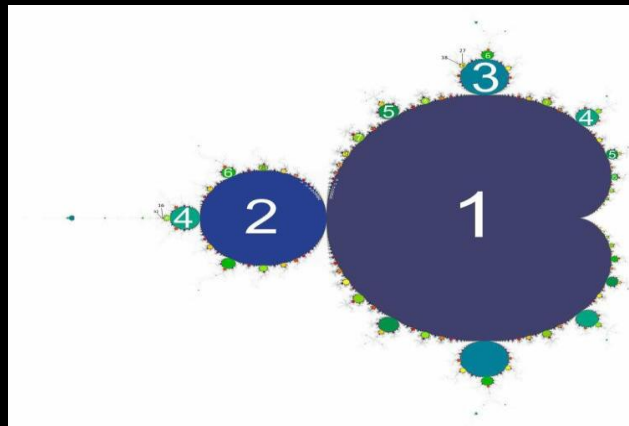
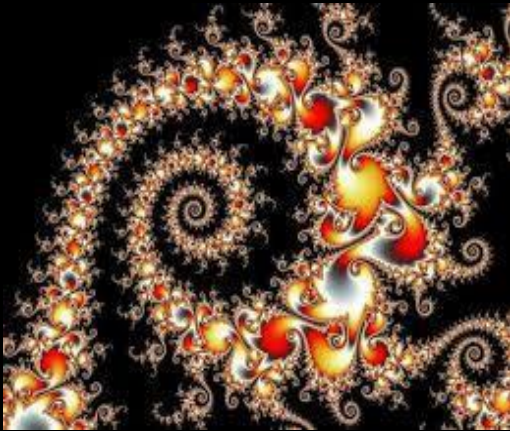
<https://www.geeksforgeeks.org/what-is-recursion/>

Refer: How do fractals play a role in the creation of realistic 3D models for computer graphics?

<https://www.quora.com/How-do-fractals-play-a-role-in-the-creation-of-realistic-3D-models-for-computer-graphics>

Mandelbrot set & Fractals in Graphics

- The Mandelbrot set has become popular outside mathematics both for its **aesthetic appeal** and as an **example of a complex structure arising from the application of simple rules**.



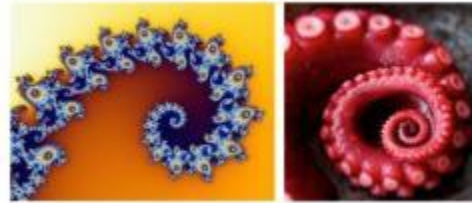
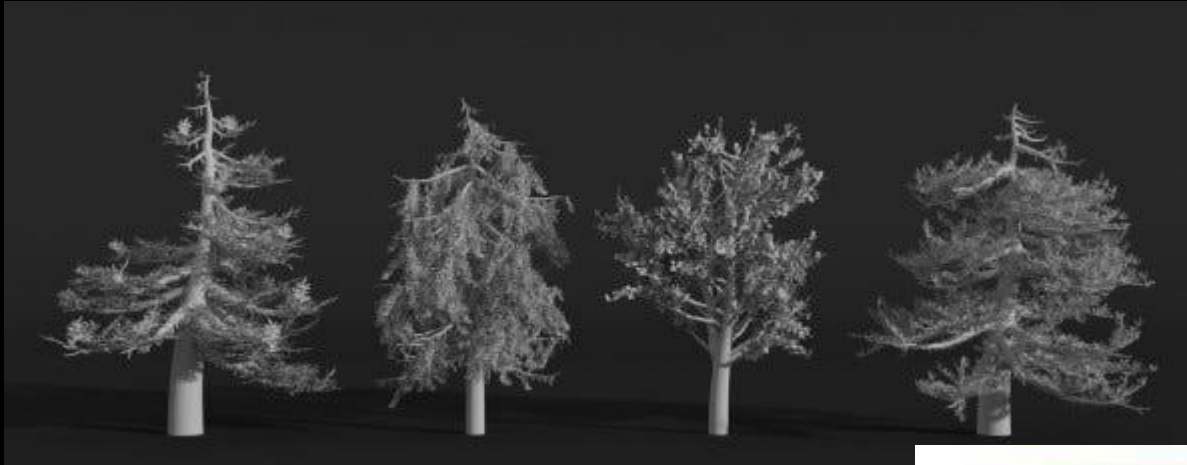
Refer : <https://math.hws.edu/eck/js/mandelbrot/MB.html>

<https://math.hws.edu/eck/js/mandelbrot/MB-info.html>

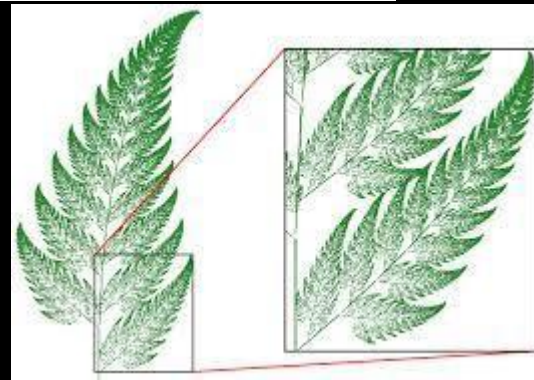
Fractal-generating software

- https://en.wikipedia.org/wiki/Fractal-generating_software
- Fractal generation used for modeling is part of realism in computer graphics.
- Fractal generating software creates mathematical beauty through visualization.
- Modern computers may take seconds or minutes to complete a single high resolution fractal image.
- Images are generated for both simulation (modeling) and random fractals for art.
- Fractal generation software can be used to mimic natural landscapes with fractal landscapes and scenery generation programs.
- Fractal imagery can be used to introduce irregularity to an otherwise sterile computer generated environment.

Fractal generated Graphics in 3D Computer Games?

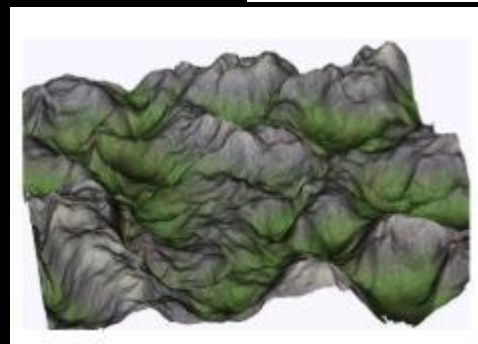
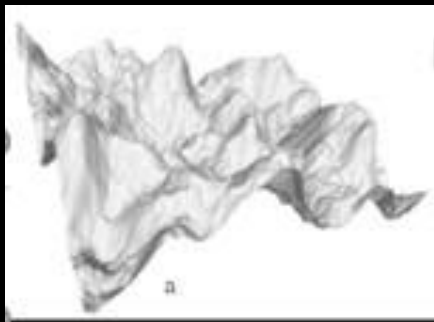
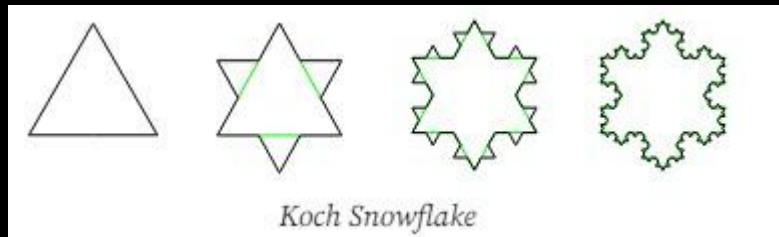
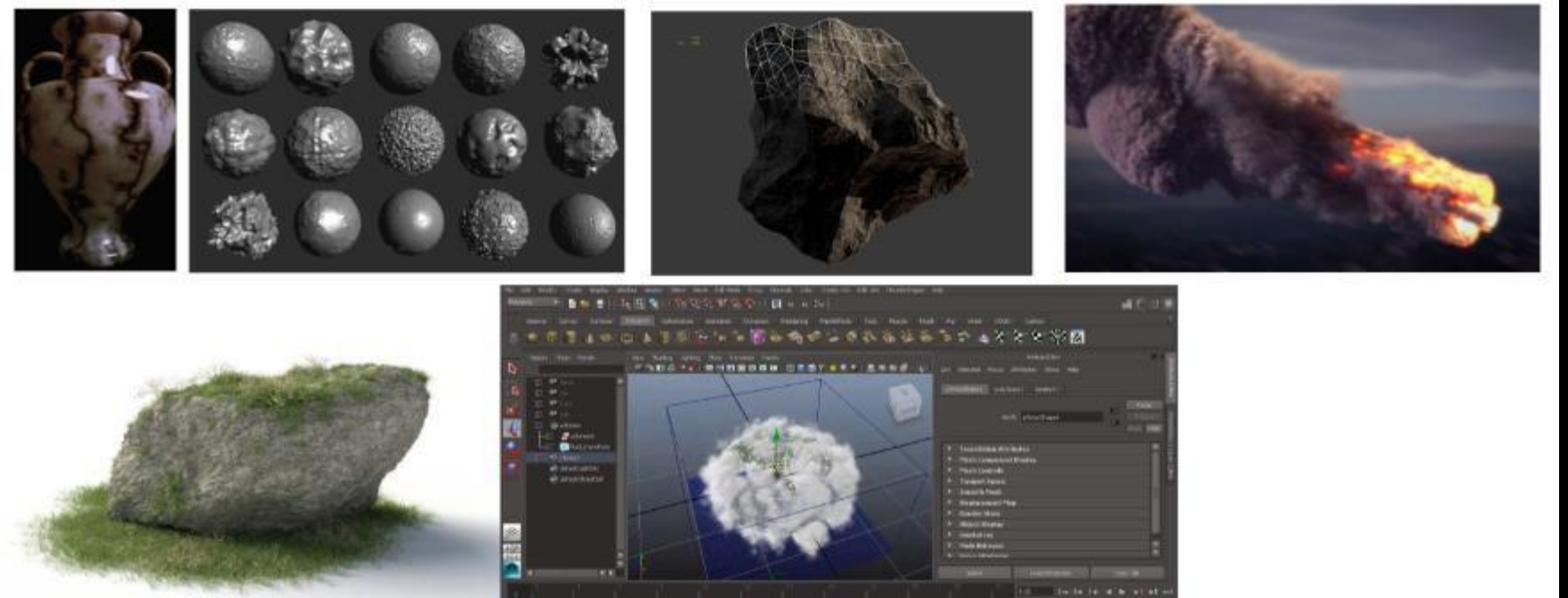


Ferns and Cactus Plants?=> self recursion



<https://remptongames.medium.com/links-awakening-and-fractal-game-design-e3af668e6be9>

Fractals



Advantages of Recursion

- Understanding types of recursion can help in **designing efficient and elegant recursive algorithms** while **considering the potential trade-offs**
- Recursion can **simplify complex problems** by breaking them down into smaller, more manageable pieces.
- Recursive **code can be more readable and easier to understand** than iterative code.
- Recursion is **essential for some algorithms and data structures**.
- Also with recursion, we can **reduce the length of code**.
- RECURSION IS AN IMPORTANT CONCEPT TO UNDERSTAND!

Disadvantages of Recursion

- Recursion can be less efficient than iterative solutions in terms of memory and performance.
- Recursive functions can be more challenging to debug and understand than iterative solutions.
- Recursion can lead to stack overflow errors if the recursion depth is too high.

Recursion – Some Best Practices

- **Identify the Base Case(s) Carefully:** Base case(s) provide the termination condition for recursion. Ensure that the base case condition is well-defined and reachable to **avoid infinite recursion**.
- **Ensure Progress Towards the Base Case:** In the recursive case(s), ensure that the problem is being **divided into smaller subproblems that lead to reaching the base case eventually**. Each recursive call should make progress toward the base case.
- **Properly Manage Memory and Resources:** Recursion may consume a significant amount of memory, especially **if the recursive calls are nested deeply**. Be mindful of memory usage and consider optimization techniques like tail recursion or **memoization when applicable**. (Memoisation covered in later lecture)
 - **What is memoization? A Complete tutorial**
 - <https://www.geeksforgeeks.org/what-is-memoization-a-complete-tutorial/>
- **Test with Different Input Sizes:** Recursion may have different performance characteristics depending on the input size. **Test your recursive function with various input sizes to identify any potential performance bottlenecks.**

Recursion – Some Common Pitfalls

- **Stack Overflow**: If the recursion **depth becomes too large**, it can result in a stack overflow error.

This **happens when the call stack, which keeps track of function calls, exceeds its memory limit.**

- Ensure that your recursive function terminates within reasonable **recursion depths** to avoid this issue.
- **Redundant or Incorrect Recursive Calls**: Be careful with the recursive function calls within the function body.

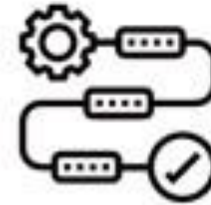
Make sure the **arguments passed to the recursive call are appropriate and lead to a valid progression toward the base case**. Incorrect or redundant recursive calls can lead to **incorrect results or infinite recursion.**

- The **base case is missing entirely**, or the problem **needs more than one base case but not all the base cases are covered.**

RECURSION

VS

ITERATION



In recursion, function calls itself directly or indirectly, thus achieving repetition

Recursion stops when the base case is reached

Recursive function stores all the steps in a memory stack

Recursion continues to generate simplified versions of the original problem until the base case is reached

Recursive code is comparatively smaller

Stack overflow error occurs which may crash the system, if the base case is not defined

Recursion is slower in execution

Best to use if problems can be divided into smaller subproblems that are similar to the original problem

Iteration specifically uses a repetition structure with conditional loop

Iteration function gets terminated when the loop condition fails

Iteration doesn't use stack memory

Iteration will continue to modify the counter until the loop continuation condition fails

Iterative code tends to be bigger in size

There will be an infinite loop if the control variable doesn't reach the termination value

Iteration is comparatively faster

Best to use if problems can be divided into smaller, repeating steps

Recursion vs Loops/Iteration

Loops

- Loop control variable
- Other Boolean event
- while
- do while
- for

Recursion

- method that calls itself
- uses an if else structure
- recursive call
- base case

A loop is memory efficient.

Recursion is memory costly.

Anything a loop can do, recursion can do as well.

There are some things recursion can do that a loop CANNOT do.

When you can, use a loop.

If a loop won't work as well, or at all, use recursion.

Both loops and recursion repeat
But in different ways
Loops are preferred
But sometimes recursion works better

Recursion vs Loops/Iteration

Why loops alone are difficult in examples like printing directory structure in a OS:

- A loop only repeats sequentially.
- A tree can branch into many subtrees.
- Recursion automatically remembers where to return using the call stack.



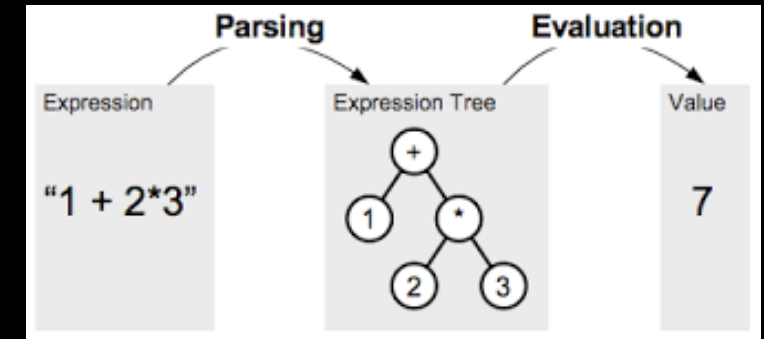
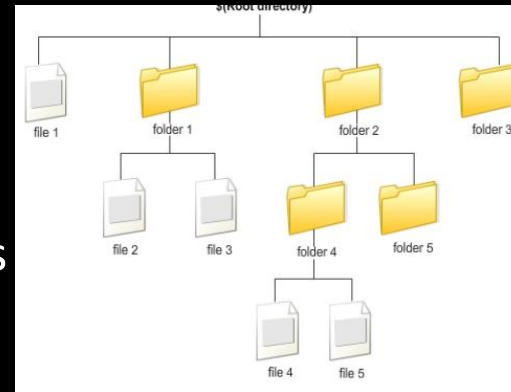
Tower of Hanoi in C++



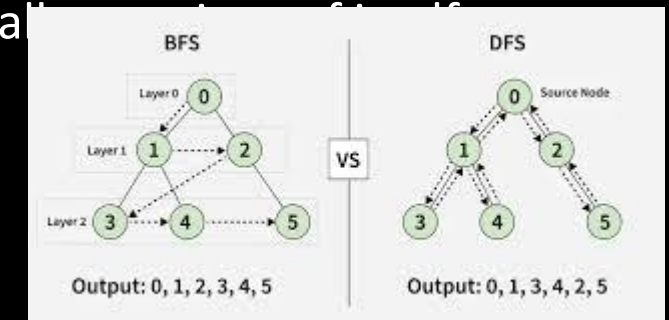
You can simulate this with loops using your own stack data structure, but pure loops alone are not enough naturally.

Further examples:

- Nested directories
- Parsing mathematical expressions
- Tower of Hanoi
- Depth First Search(DFS) in graphs



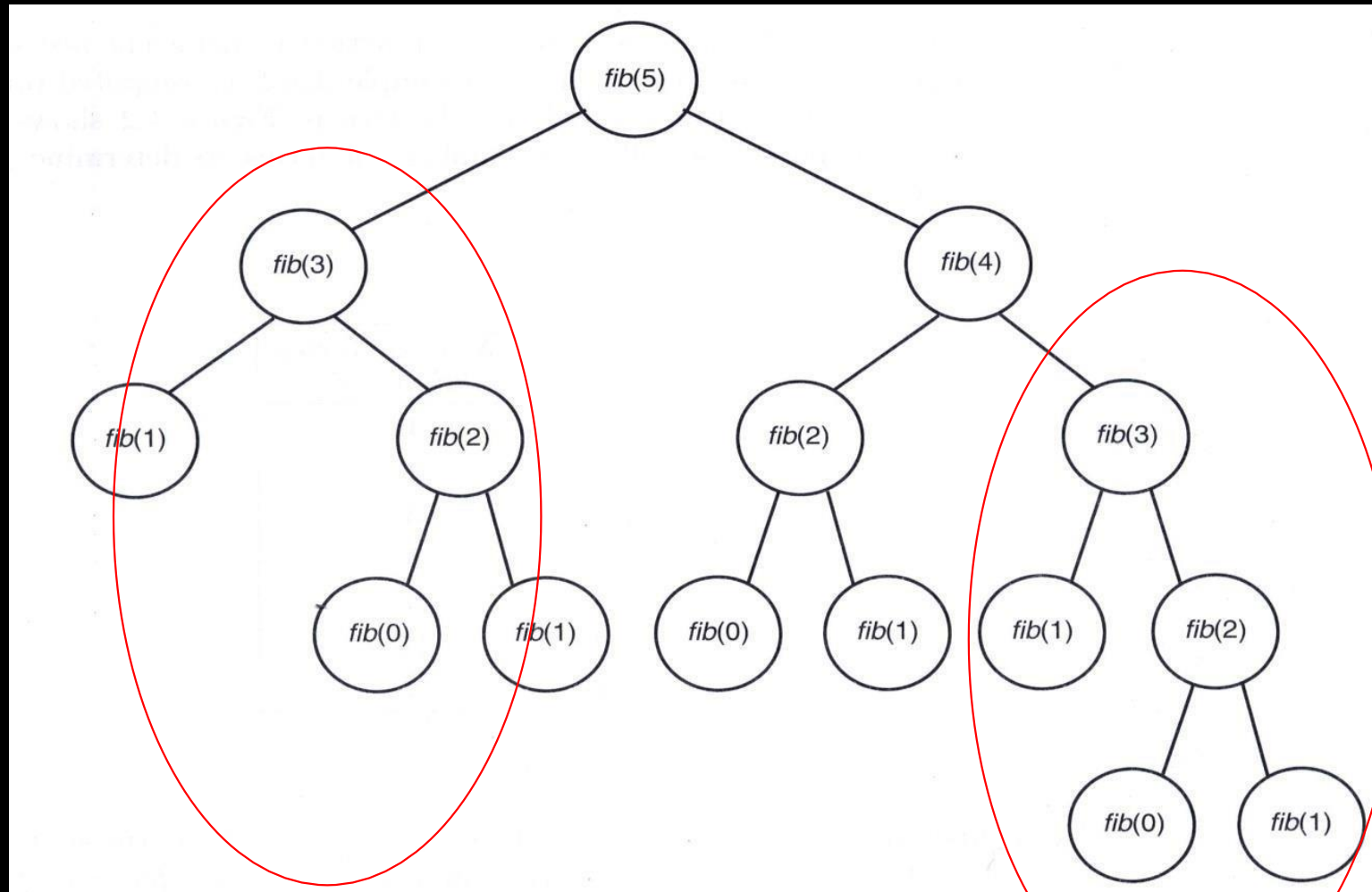
These problems are naturally recursive because the problem contains smaller instances of itself.



1st Solution to the n-th Fib. Number (Recursive version)

```
int fib (int n) {  
    if (n <= 1)  
        return n;  
    else  
        return fib(n-1) + fib(n-2);  
}
```

The Recursion Tree when Computing the Fifth Fibonacci Term.



2nd Solution to the n-th Fib. Number (Iterative version)

```
int fib2 (int n) {  
    index i;  
    int f[0..n];  
  
    f[0] = 0;  
    if (n > 0) {  
        f[1] = 1;  
        for (i = 2; i <= n; i++)  
            f[i] = f[i-1] + f[i-2];  
    }  
    return f[n];  
}
```

Questions?

